

数字系统设计

Digital System Design

-----微控制器之1---算法状态与控制器

王维东

Weidong Wang

浙江大学信息与电子工程学院

College of Information Science & Electronic Engineering

信息与通信工程研究所

Zhejiang University

Spring 2026 – Lec. #08

任课教师

- 王维东
- 浙江大学信息与电子工程学院, 信电楼306
- 邮箱: wdwang@zju.edu.cn
- College of Information Science & Electronic Engineering
- Zhejiang University, Hangzhou, 310027
- Tel: 86-571-87953170 (O)
- Mobile: 13605812196

TA:

钟睿, 13687791509; 22331166@zju.edu.cn;

Office Hours: 玉泉信电楼 308室（可以钉钉、微信或邮件联系）.

Prerequisites预修课程

- 电子电路基础
- 电子线路
- C语言
- How to learn this Course?
 - Not only listening, thinking and waiting
 - But Exercise, Simulation, Practice!

课程简介

- 课程代码: **671C0050**
- 参考书
 - 阎石,《数字电子技术基础》,第6版,高等教育出版社,2016.
 - 王金明著,《数字系统设计与Verilog HDL》,电子工业出版社,第8版
 - 《数字系统设计-补充讲义》,浙江大学自编讲义
 - Stanford 大学 108A课程notes.
 - R.H.Katz, G.Borriello, *Contemporary Logic Design*, second edition,电子工业出版社,2005.
 - M.M.Mano, 数字设计(第四版),电子工业出版社,2010.
 - » <http://www.prenhall.com/mano>

Other Course Info

- **Website:**

- <http://mypage.zju.edu.cn/wdwd/教学工作/>
- **学在浙大: 数字系统设计**
- Check frequently

- **答疑**

- 玉泉信电楼308室/周一周四下午2:30-5:30
- 上课课间、课后均可
- Email, 钉钉/微信群/数字系统设计, 短信均可

Grading (考核)

Final grades will be computed approximately as follows:

- 平时（含课程作业、期中考试、小测验、Project、出勤等） **40%**

- 期中考试 **10%**
- Homework Sets **15% (12~20)%**
- Project **5% (5~7)%**
- Class Room Check+小测验等 **10% (7~10)%**
- 作业每周一上交, **截止期为课后一周内**有效。

- **Project**

- Project-1必做 (1 members)
- Project-2可选(成绩加分1%~5%, 但不超过平时成绩范围)

- Finial Exam期末**闭卷**考试 - **60%**

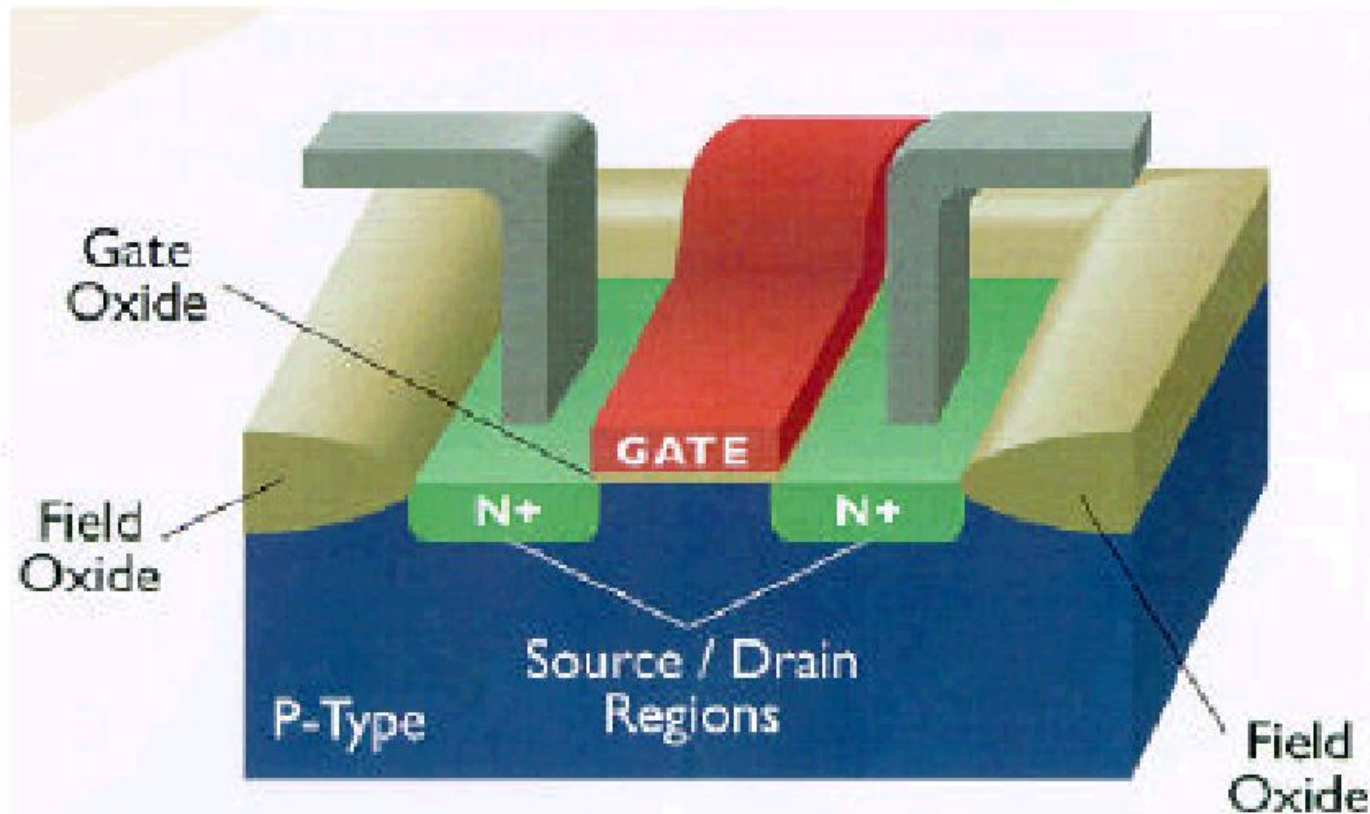
授课时间和地点:

- 2025年春夏学期,
 - 周一上午, 第1~2节(8:00-9:35)
 - 星期四上午, 第6~7节(13:25-13:00)
- 地点: 紫金港西2-415(多)
- <http://mypage.zju.edu.cn/wdwd/教学工作/>
- 学在浙大/数字系统设计
- 钉钉/微信/数字系统设计-答疑

课程结构

- 数字理论知识(必备)
 - 数字系统和编码、逻辑代数、门电路
- 数字电路分析与设计
 - 组合逻辑电路
 - 触发器、半导体存贮器、可编程器件
 - 时序逻辑电路
- 脉冲电路与接口
- 控制器与数字系统
 - 状态机
 - 控制器
 - 微码控制器
 - 测试和验证
- 微处理器简介
 - 指令集
 - **4位CPU**介绍

复习：MOS管结构和符号



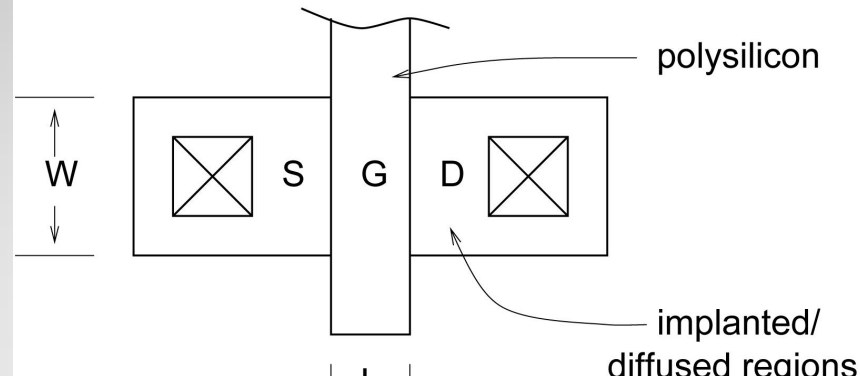
$V_{GS(th)}$ 称为MOS管的开启电压

$V_{GS}=0$ 漏极和源极之间相当于两个PN结背向地串联，
所以D-S间不导通, $i_D=0$

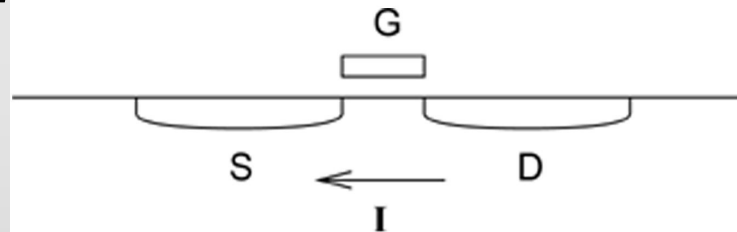
CMOS Devices

- MOSFET (Metal Oxide Semiconductor Field Effect Transistor).

Top View



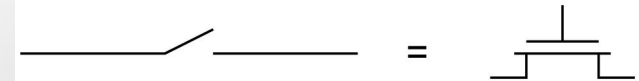
Cross Section



The gate acts like a capacitor. A high voltage on the gate attracts charge into the channel. If a voltage exists between the source and drain a current will flow. In its simplest approximation the device acts like a switch.

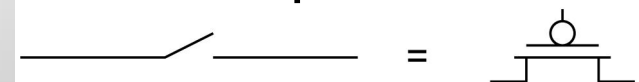
nFET

$V_{gs} = '0'$



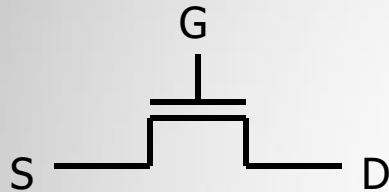
pFET

$V_{gs} = '1'$



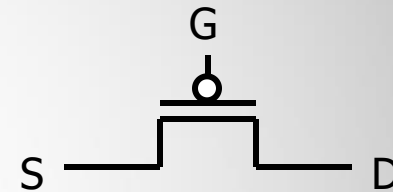
What Complementary about CMOS?

- Complementary devices work in pairs



n-channel

open when voltage at G is low
closes when:
 $\text{voltage}(G) > \text{voltage}(S) + \varepsilon$



p-channel

closed when voltage at G is low
opens when:
 $\text{voltage}(G) < \text{voltage}(S) - \varepsilon$

重要补充:

新第8章 微控制器设计

-----算法状态机

8.1 概述

- 数字系统
 - 二进制信息
 - 存储在寄存器或存储器中
 - 数据信息和控制信息
- 数据
 - 要处理的离散信息
 - 算术、逻辑、移位等微操作
 - 执行部件：加法器、译码器、乘法器、计数器和移位寄存器等
- 控制信息
 - 命令信号
 - 协调和实施各种微操作序列

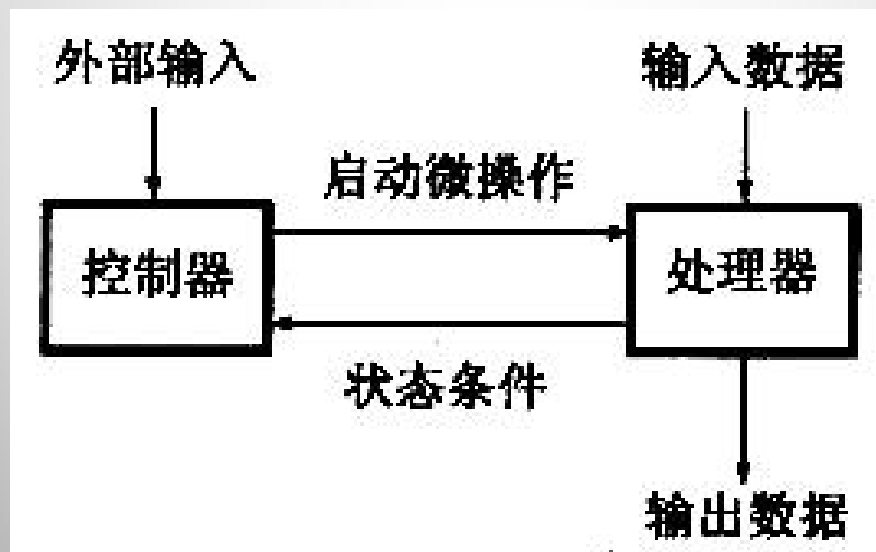
控制器和数据处理器的关系

- “算法”

- 是指一个问题的求解过程，可以通过有限个定义的程序性步骤来说明。

- 同步数字系统

- 只有当寄存器由控制信号选通时，**时钟脉冲**才能改变其状态
- 各个选择变量和各个寄存器的选通输入是由控制器产生的。



微操作 与流程图

- 微操作

- 产生微操作控制序列信号的控制器是一个时序电路
- 控制器是为启动数字系统中数据处理器的微操作提供控制信号时间序列的
- 拟定**控制序列**的同时，**决定**每个控制状态所**启动**的微操作

- 时间图或流程图

- 通过时间图或者流程图来描述系统的操作序列比较直观、清楚
- 时间图
- 流程图

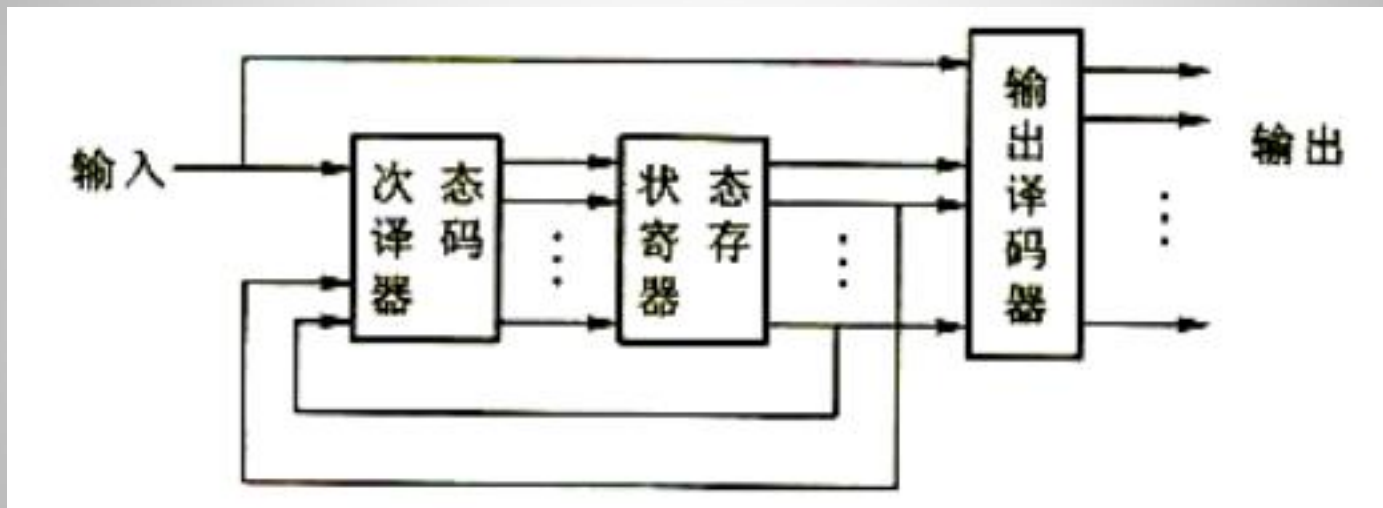
控制器的逻辑设计

- 特殊的方法
 - 建立在经典**时序逻辑**方法和**寄存器**传送方法基础上的。
- 四种方法
 - (1)每个状态一个触发器/**one-hot**;
 - (2)序列寄存器—译码器法;
 - (3)**PLA**控制法;
 - (4)微程序控制法;

8.2 状态机设计

- 状态机

- 是指按有序方式**遍历**预先确定的状态序列的数字逻辑功能电路。



- 状态机中的状态寄存器记忆了状态机的内部状态

状态机

- 状态机主要完成两种基本操作

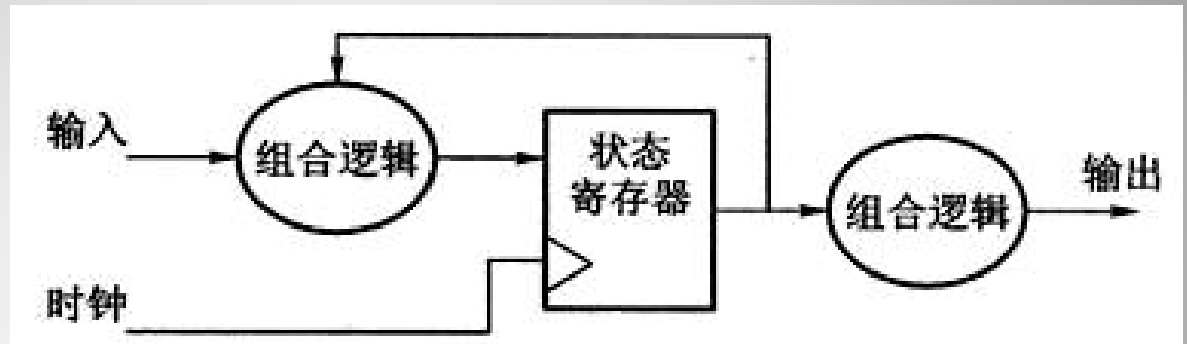
- 1. 状态机内部状态转换。遍历某一确定的状态序列，其中次态由次态译码器根据现态和输入条件来确定。
- 2. 根据状态变化(称为状态转移)产生输出信号。输出译码器根据现态和输入条件可确定输出信号。

- 状态机的两种基本类型

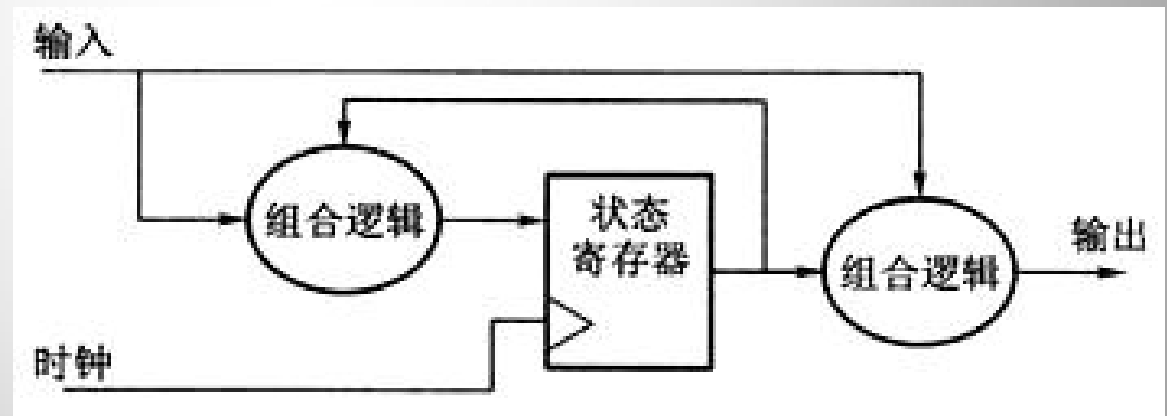
- 莫尔(Moore)状态机
 - » 莫尔状态机的输出只是当前状态的函数
- 米里(Mealy)状态机
 - » 米里状态机输出是当前状态和输入信号的函数

有限状态机(FSM)

- 输出取决于过去状态和当前状态的状态机
- (a) Moore机



- (b) Mealy机



- 统一优化方法
 - 消除毛刺
 - » Gray码状态编码
 - » 输出加入寄存器
 - 状态完全编码
 - 同步时序电路

State Elements

Always blocks are the only way to specify the "behavior" of state elements. Synthesis tools will turn state element behaviors into state element instances.

D-flip-flop with synchronous set and reset example:

```
module dff(q, d, clk, set, rst);  
  input d, clk, set, rst;  
  output q;  
  reg q;
```

```
  always @(posedge clk)
```

```
    if (rst)
```

```
      q <= 1'b0;
```

```
    else if (set)
```

```
      q <= 1'b1;
```

```
    else
```

```
      q <= d;
```

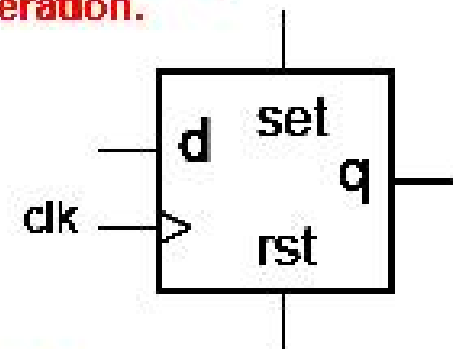
```
endmodule
```

keyword

"always @ (posedge clk)" is key to flip-flop generation.

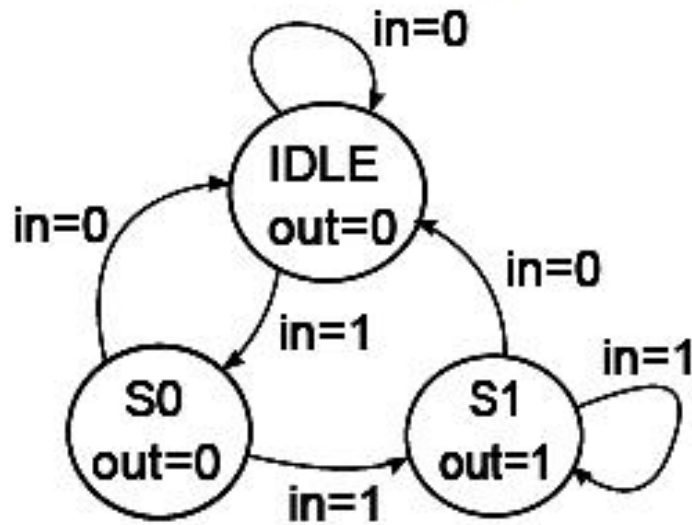
This gives priority to reset over set and set over d.

On FPGAs, maps to native flip-flop.



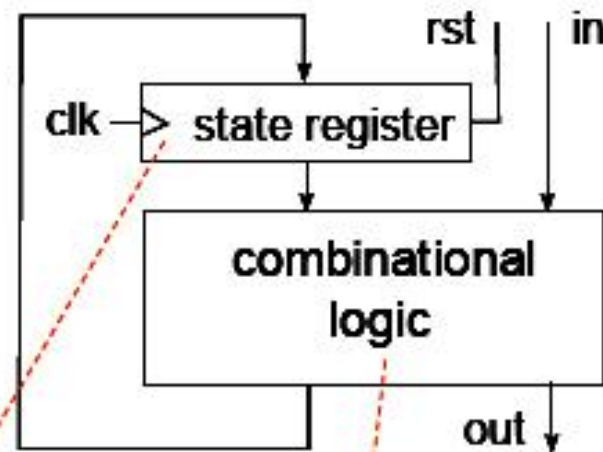
Finite State Machines

State Transition Diagram



Holds a symbol to keep track of which bubble the FSM is in.

Implementation Circuit Diagram



CL functions to determine output value and next state based on input and current state.

$out = f(in, \text{current state})$

$\text{next state} = f(in, \text{current state})$

What does this one do?

Did you know that every SDS is a FSM?

FSMs (cont.)

```
// always block for combinational logic portion
always @(state or in)
case (state)
// For each state def output and next
```

```
  IDLE : begin
        out = 1'b0;
        if (in == 1'b1) next_state = S0;
        else next_state = IDLE;
```

```
  end
```

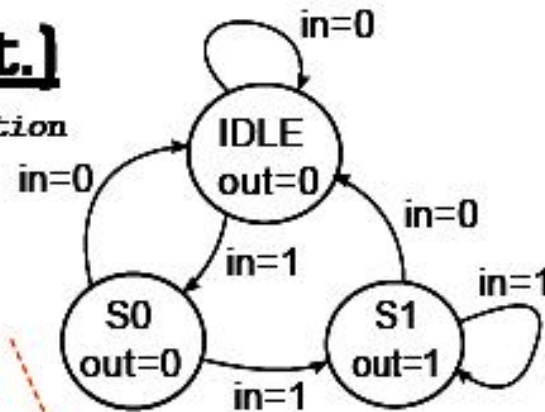
```
  S0 : begin
        out = 1'b0;
        if (in == 1'b1) next_state = S1;
        else next_state = IDLE;
```

```
  end
```

```
  S1 : begin
        out = 1'b1;
        if (in == 1'b1) next_state = S1;
        else next_state = IDLE;
```

```
  end
```

```
default: begin
        next_state = IDLE;
        out = 1'b0;
      end
```



Each state becomes
a case clause.

For each state define:
Output value(s)

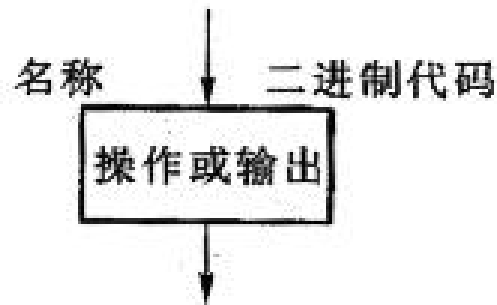
State transition

Use "default" to cover unassigned state.
Usually unconditionally transition to reset state.

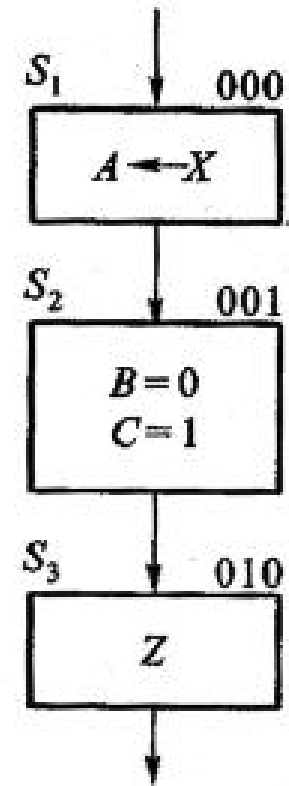
状态机有三种表示方法：状态图、状态表和流程图。

8.2.2 算法状态机图ASM

- Algorithmic State Machine Chart
- ASM图符号



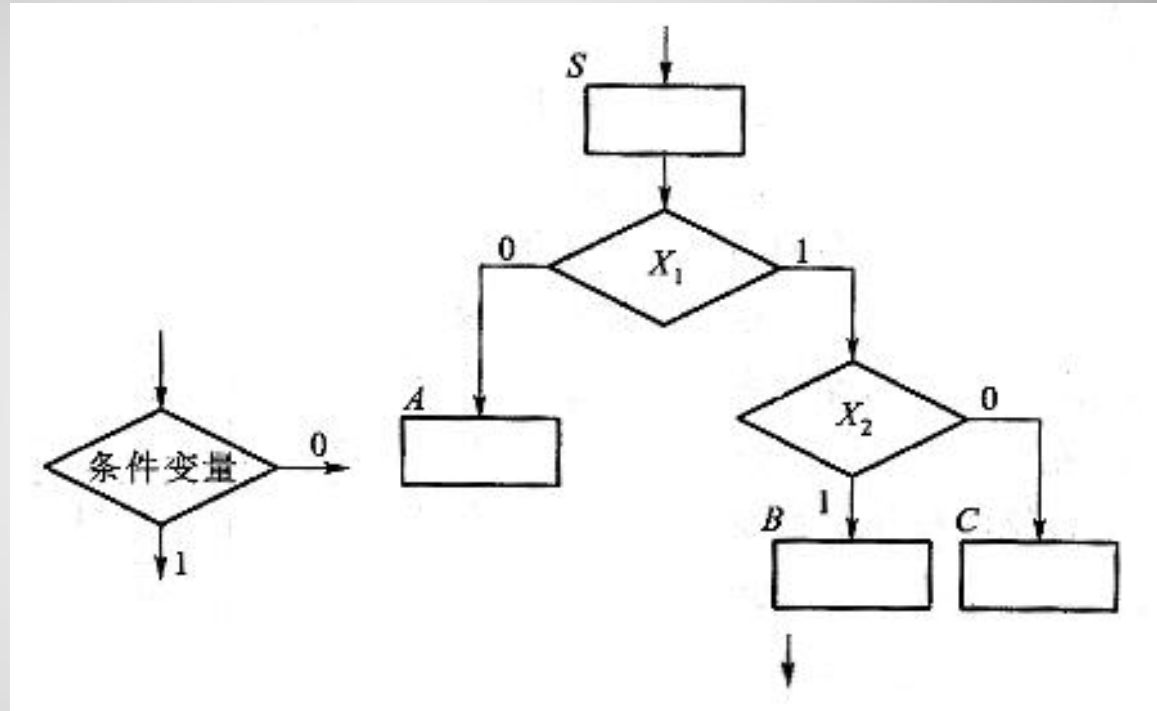
(a) 状态框符号



(b) 状态框组成的ASM图实例
ASM图状态框

ASM图

- 判断框

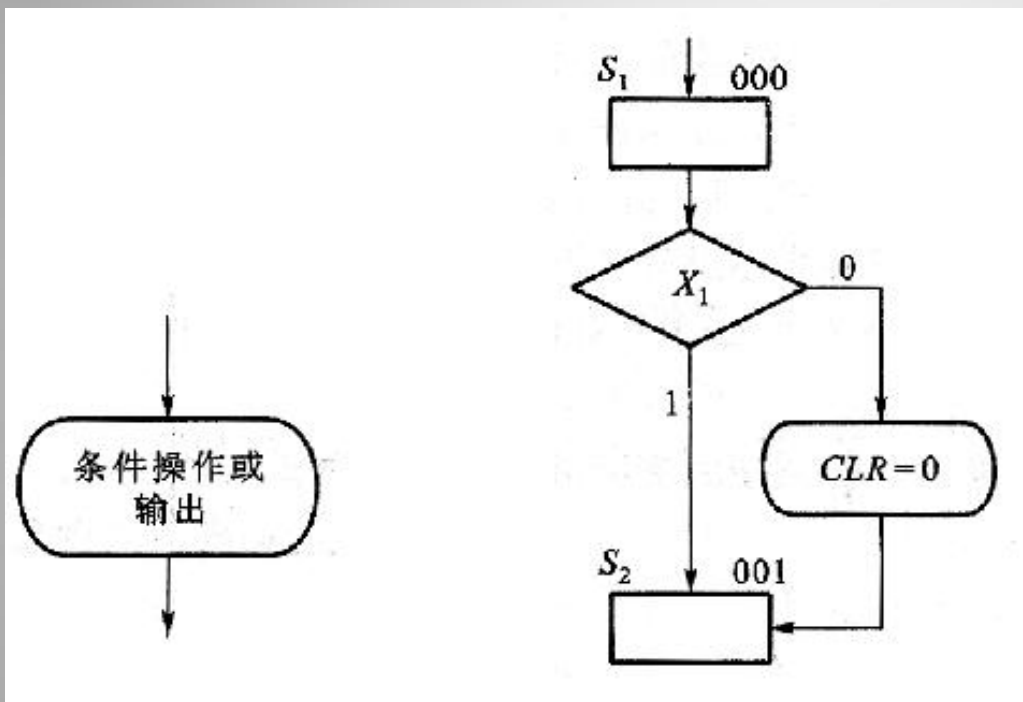


(a)判断框符号

(b)判断框构成的ASM图实例
ASM图判断框

ASM图

- 条件框



(a)条件框符号

(b)条件框构成的ASM图实例
ASM图条件框



开始块和结束块

ASM图的时间划分

- **ASM块**

在ASM图中，一般把一个状态框和它下属的几个判断框和条件框，称为一个ASM块

—一个时钟周期内系统的工作情况

—包括数据子系统和控制器两个方面

—即在当前状态及条件下，数据子系统所完成的各种操作以及控制器转换的后续状态。

- **ASM图是*按时钟的节拍*描述整个数字系统的操作**

—所有的操作和状态转换都发生在时钟的同一个跳变边沿

- **ASM图与流程图的区别**

—流程图中表明事件的顺序

—而ASM图还进一步表明了状态的顺序

ASM图的建立

- 建立系统的**ASM图**的原则：

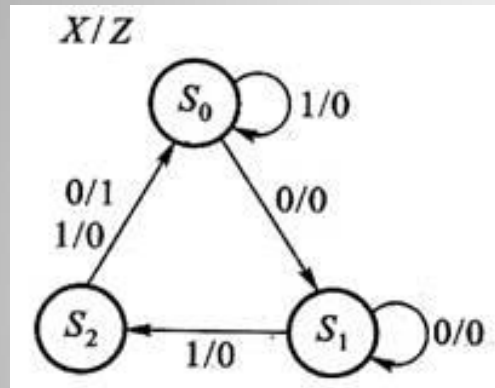
- 1. 流程图中的工作块基本上对应了**ASM图**中的状态框。
 - » 但如果工作块的操作不能在一个**CP内**同时进行，在**ASM图**中就必须将其分为几个状态框，在这几个状态之间实现无条件转移。
- 2. 流程图中的判断块基本上对应了**ASM图**中的判断框。
 - » 但如果判断条件是上个操作的结果，那么在**ASM图**中应在此判断框前增加一个状态框
 - » 因为在**ASM图**中，各个**ASM块**的执行时间是同一个**CP**的有效边沿
 - » 如果不增加一个状态框，则判断条件对应于前一个**CP**的工作块的操作结果。
- 3. 一般在**ASM图**的最上层加一个起始状态。

建立ASM图

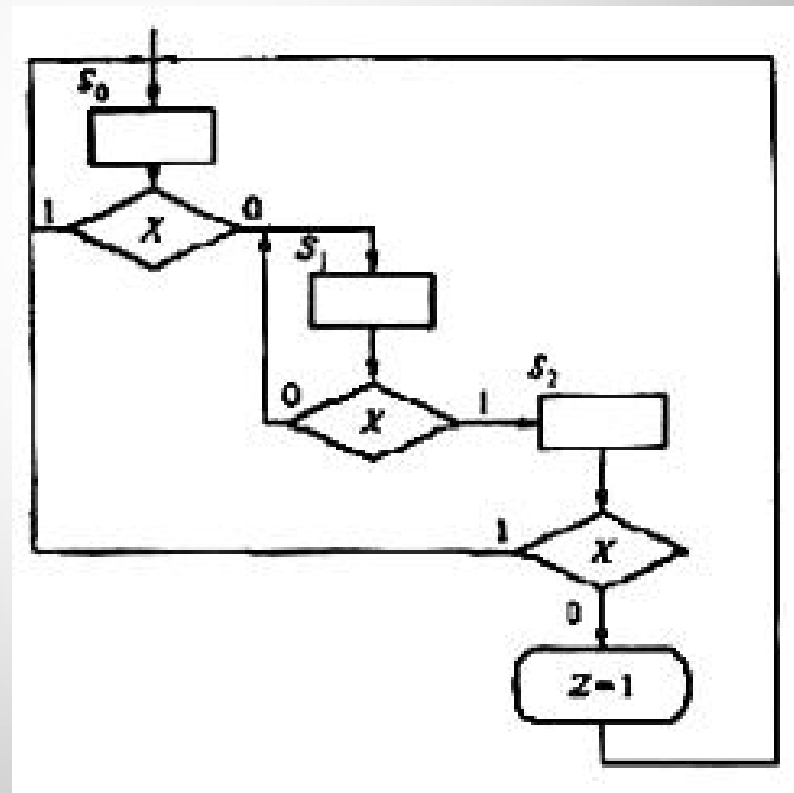
- **【例8.1】** 串行数据序列是每个时钟周期传送一个数据0或1的数据流。设 x 为输入的串行数据序列。当检测到数据流中出现所需的010数据时，使检测器的输出 Z 为1。试画出其ASM图。
- **解** 这是一个米里型时序电路，其输出 Z 与输入信号有关。
 - 设 S_0 为起始态，
 - S_1 为收到 X 序列里的0后的状态，
 - S_2 为收到 X 序列里0-1后的状态，若接着后面 X 输入的是0的话，则输出 $Z=1$ ，并且状态返回 S_0 。

转换图

- 状态转换图



- ASM图



【例8.2】

- 试设计一个米里型同步模4可逆计数器，推导出该计数器的算法流程图。
- 解
 - 计数器功能分析：
 - (1)当控制信号 $x=0$ 时
 - » 计数器对输入脉冲CP进行累加计数，并在累加4个脉冲时，输出进位 $z=1$ 。
 - (2)当控制信号 $x=1$ 时
 - » 计数器对输入脉冲CP进行累减计数，并在累减4个脉冲时，输出借位 $z=1$ 。



■图7.12(a) 模4可逆计数器的示意图。

解

- 根据题意，模4计数器有两个二进制寄存器， $R=R_1R_2$ 。
- 模4计数器算法的具体做法为：
- $$x = \begin{cases} 0 & R \leftarrow R_i + 1 & \text{当 } i = 3 \text{ 时, } Z = 1 (i = 0 \sim 3) \\ 1 & R \leftarrow R_i - 1 & \text{当 } i = 0 \text{ 时, } Z = 1 (i = 0 \sim 3) \end{cases}$$

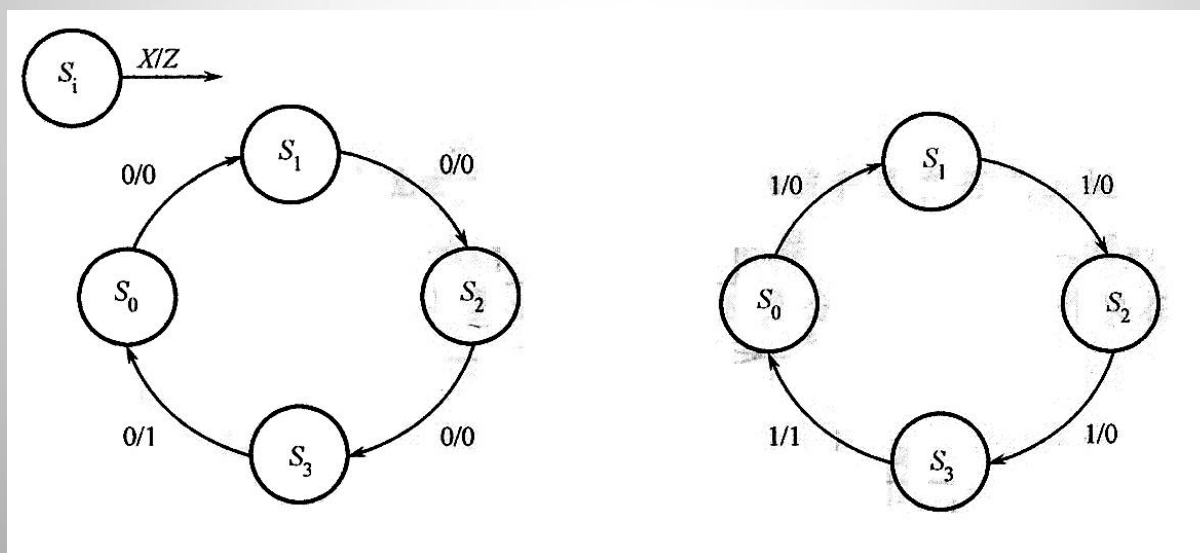


图7.12(b) 模4可逆计数器状态图

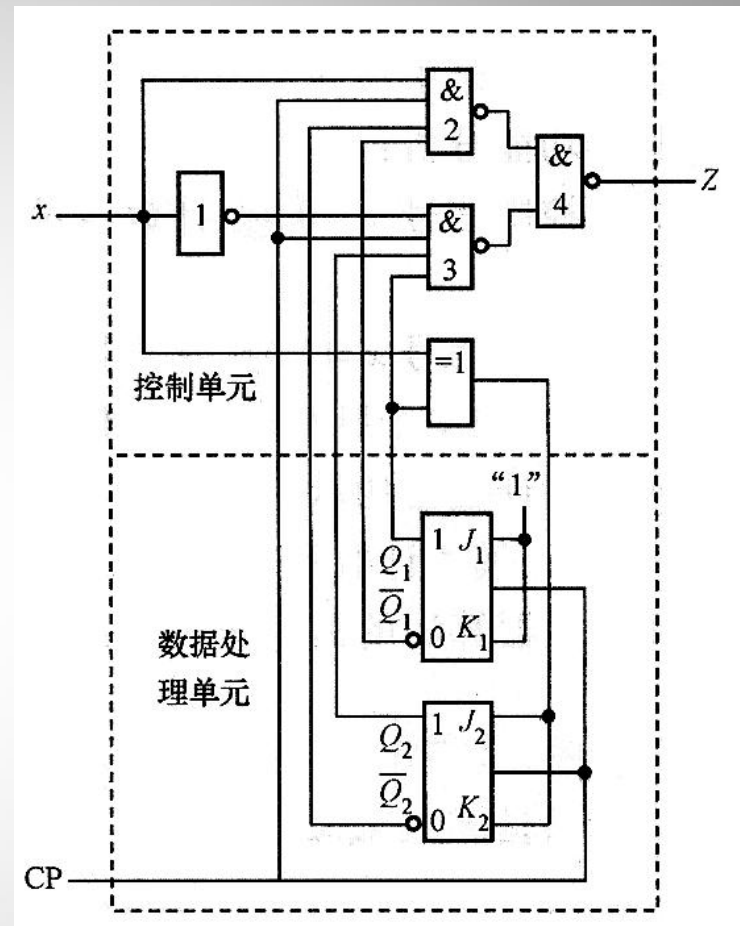
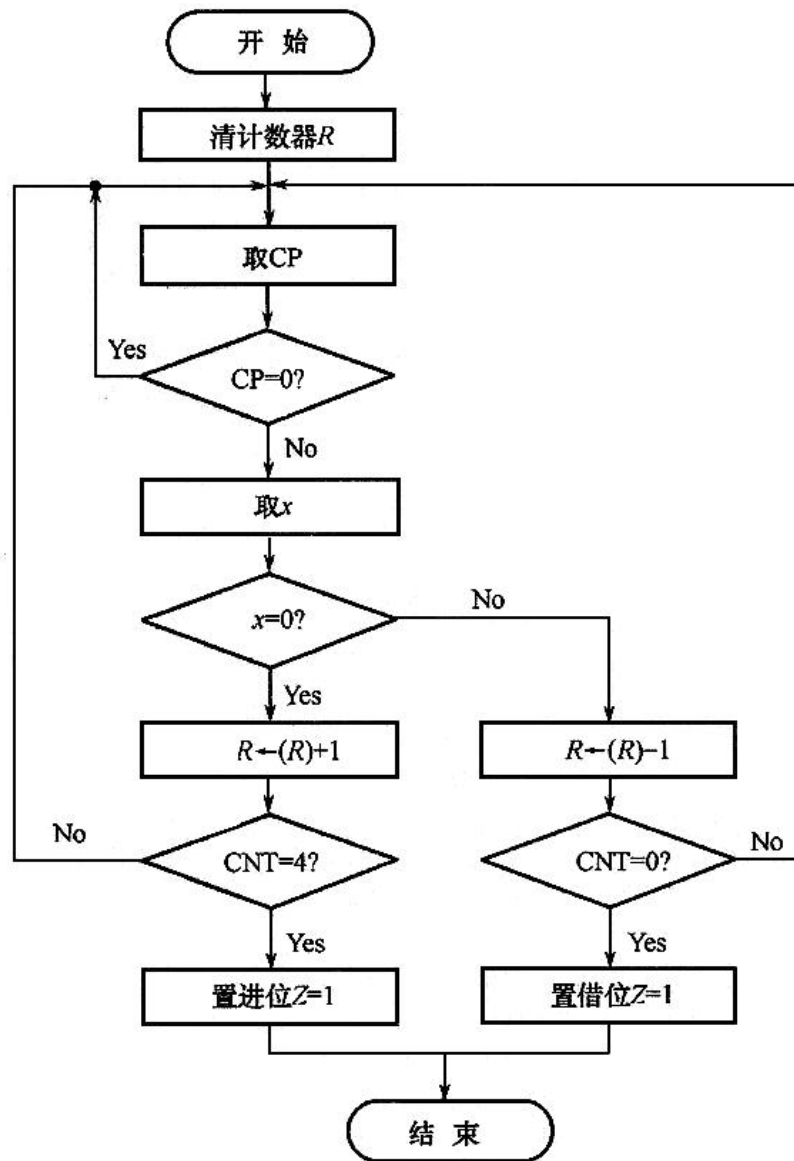
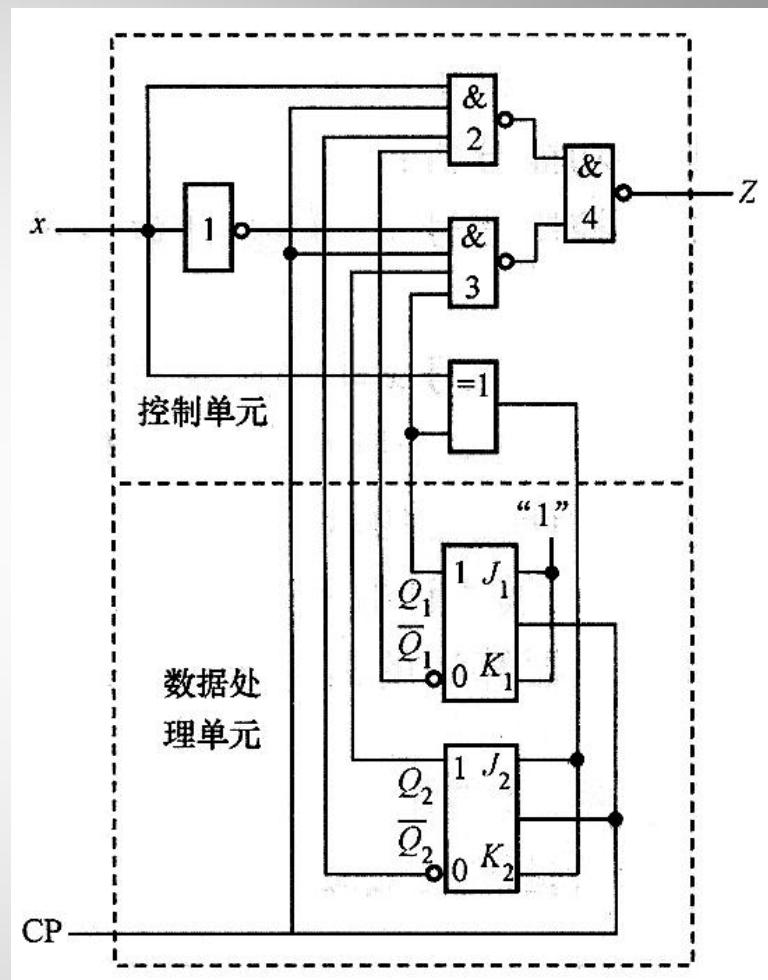


图7.12(d) 模4可逆计数器逻辑图

分析小结

从图8.12(d)可以清楚地看到:

- 这个简单的数字系统由**控制单元**和**数据处理单元**两部分组成。
- **控制单元**的基本功能在于对指令和数据流实施时间和空间上的**正确控制**。
- **数据处理单元**是在控制信号的控制下, 有序地实现**数据处理**操作。



8.2.3 状态机设计

- 状态机的设计四个步骤：
 - 1. 确定任务。
 - » 首先要对实际应用问题进行分析和归纳，以确定控制系统的任务以及要实现的功能。
 - 2. 把设计任务转化为状态图。
 - » 即列出采用的状态机全部可能的状态，并对每一个状态进行状态编码及定义相应的状态转换条件。
 - 3. 根据状态图(或把状态图转化为状态表，并对状态图和状态表进行必要的简化处理)和输出函数，画出状态转移图。
 - 4. 建立激励函数和输出函数，画出逻辑电路。

图8.13 某数字系统的控制器ASM图

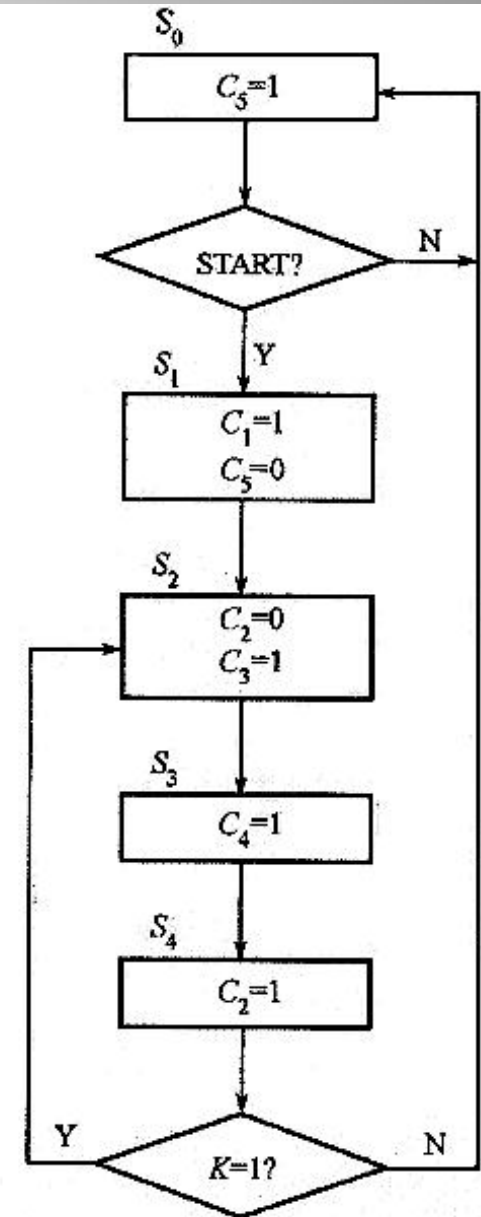
实例

【例8.3】已知某数字系统的控制器ASM图如图8.13所示，试设计对应的逻辑电路图。

解

1. 对ASM图进行状态分配

- 图中的S0、S1、S2、S3和S4五个状态
- 用三位状态变量Q2、Q1、Q0的二进编码

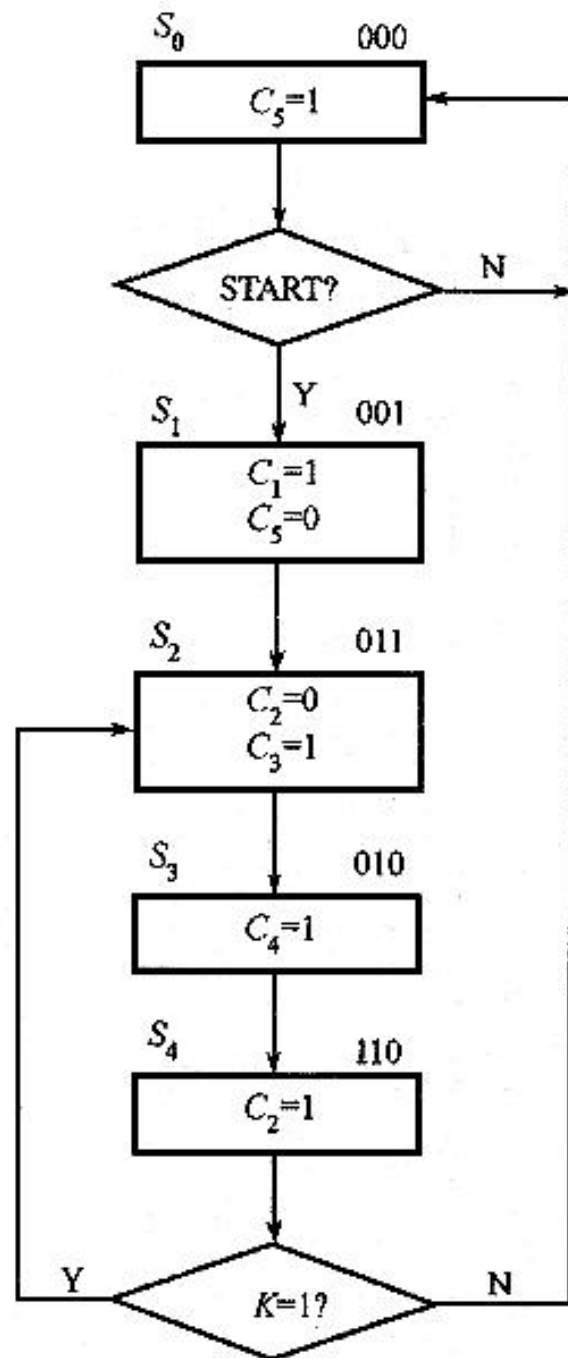


解-2

a).得编码的**ASM**图如右图

b).得状态分配图

$Q_2 \backslash Q_1 Q_0$		00	01	11	10
		0	1	1	0
Q_2	0	S_0	S_1	S_2	S_3
	1	X	X	X	S_4



解-3

2. 建立激励表

1) 状态寄存器选择D触发器

2) 建立三个D触发器激励函数

状态S0(000)有状态分支;

■ 当START到来时, 状态S0转换到状态S1(001),

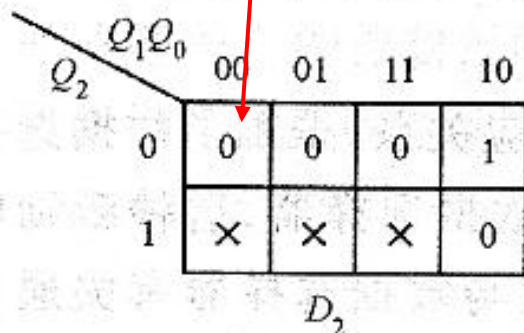
■ 否则状态S0保持不变。

■ Q2和Q1总保持为0,

■ D2和D1激励函数卡诺图上与状态S0对应的小方格中均填写0

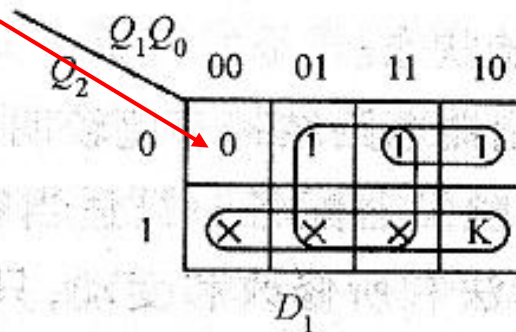
■ 而S0在满足START=1时, D0由0→1, 否则保持0。

■ D0卡诺图上状态S0对应的小方格中应填写D0置1的条件START。



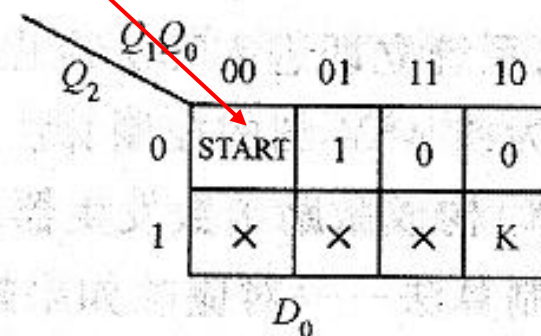
$Q_2 \backslash Q_1 Q_0$	00	01	11	10
0	0	0	0	1
1	×	×	×	0

D_2



$Q_2 \backslash Q_1 Q_0$	00	01	11	10
0	0	1	1	1
1	×	×	×	K

D_1



$Q_2 \backslash Q_1 Q_0$	00	01	11	10
0	START	1	0	0
1	×	×	×	K

D_0

2. 建立激励表

2) 建立三个D触发器激励函数

- 当START=0时, $D_2=D_1=D_0=0$, 下一个时钟, $Q_2=Q_1=Q_0=0$;
 - 控制器保持在状态S0;
- 当START=1时, $D_2=D_1=0$, 而 $D_0=1$, 下一时钟, 则 $Q_2Q_1Q_0=001$
 - 控制器由状态S0转换为状态S1
- ASM图中 $S1 \rightarrow S2$, $S2 \rightarrow S3$, $S3 \rightarrow S4$ 均为无判别条件的状态转换
 - 状态(S1、S2、S3)相应的小方格, 填入次态的二进制编码

$Q_2 \backslash Q_1Q_0$		00	01	11	10
0		0	0	0	1
1		×	×	×	0

D_2

$Q_2 \backslash Q_1 Q_0$		00	01	11	10
0	0	1	1	1	
1	×	×	×	K	

D_1

$Q_2 \backslash Q_1 Q_0$		00	01	11	10
0	START	1	0	0	
1	×	×	×	K	

D_0

2) 建立三个D触发器激励函数

状态 $S_4(110)$ 的分支情况

- 当 $K=1$ 时, 转换为状态 $S_2(011)$, 即 Q_2 由 $1 \rightarrow 0$, Q_1 保持, Q_0 由 $0 \rightarrow 1$;
- 当 $K=0$ 时, 转换为状态 $S_0(000)$, 即 Q_2 由 $1 \rightarrow 0$, Q_1 由 $1 \rightarrow 0$, Q_0 保持为 0 ;
- 在 D_2 卡诺图上与状态 S_4 对应的小方格中填写 0 。
- 因为次态 S_2 和 S_0 的 Q_2 均为 0 ,
- D_1 卡诺图上对应方格中填写 K ,
- D_0 卡诺图上对应小方格中也填写 K 。
- 得 D_2 、 D_1 、 D_0 的激励函数卡诺图

Q_2	Q_1Q_0			
	00	01	11	10
0	0	0	0	1
1	×	×	×	0

D_2

Q_2	Q_1Q_0			
	00	01	11	10
0	0	1	1	1
1	×	×	×	K

D_1

Q_2	Q_1Q_0			
	00	01	11	10
0	START	1	0	0
1	×	×	×	K

D_0

- 由卡诺图求得激励函数

$$D_2 = \overline{Q_2}Q_1\overline{Q_0} \quad D_1 = \overline{Q_2}K + \overline{Q_2}Q_1 + Q_0$$

$$D_0 = \overline{Q_1}START + Q_2K + \overline{Q_1}Q_0$$

- **3. 输出函数方程**

- 在ASM图中，每个状态块中表明了该状态的输出，条件输出由椭圆块表示
- 注意的是输出信号极性高电平有效，还是以低电平有效。
- 所有输出均为无条件输出，信号持续时间只与状态有关。

$$C_5 = S_0 = \overline{Q_2}\overline{Q_1}\overline{Q_0} \quad C_4 = S_3 = \overline{Q_2}Q_1\overline{Q_0}$$

$$C_3 = S_2 = \overline{Q_2}Q_1Q_0 \quad C_2 = S_4 = Q_2Q_1\overline{Q_0}$$

$$C_1 = S_1 = \overline{Q_2}\overline{Q_1}Q_0$$

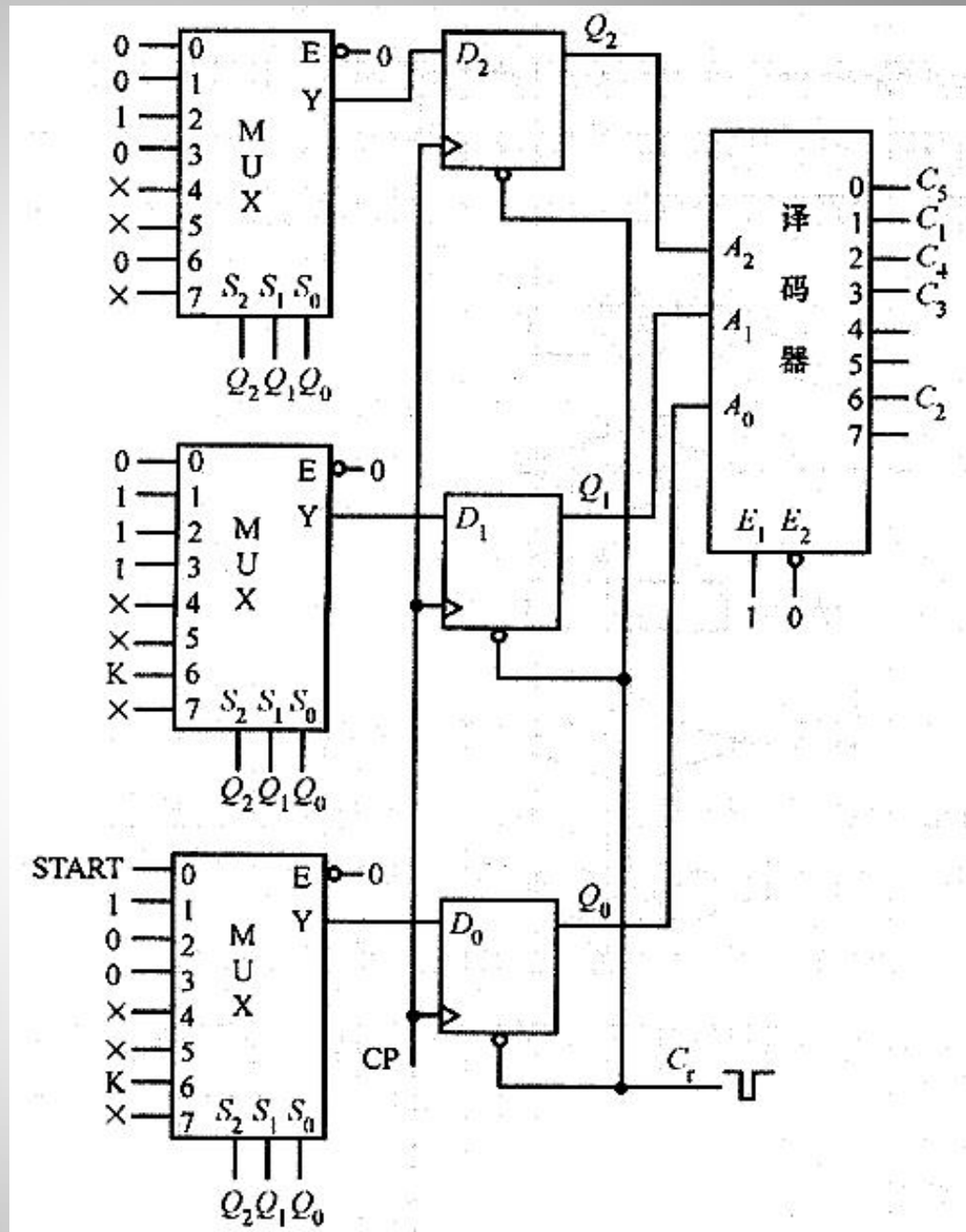
4. 逻辑电路图

• 图中

- **START**是系统外部的输入，已经同步化处理；
- **K**是数据处理单元送来的工作情况反馈信号；
- **Cr**信号为异步复位信号，开机复位，系统立即进入算法流程的初始状态。

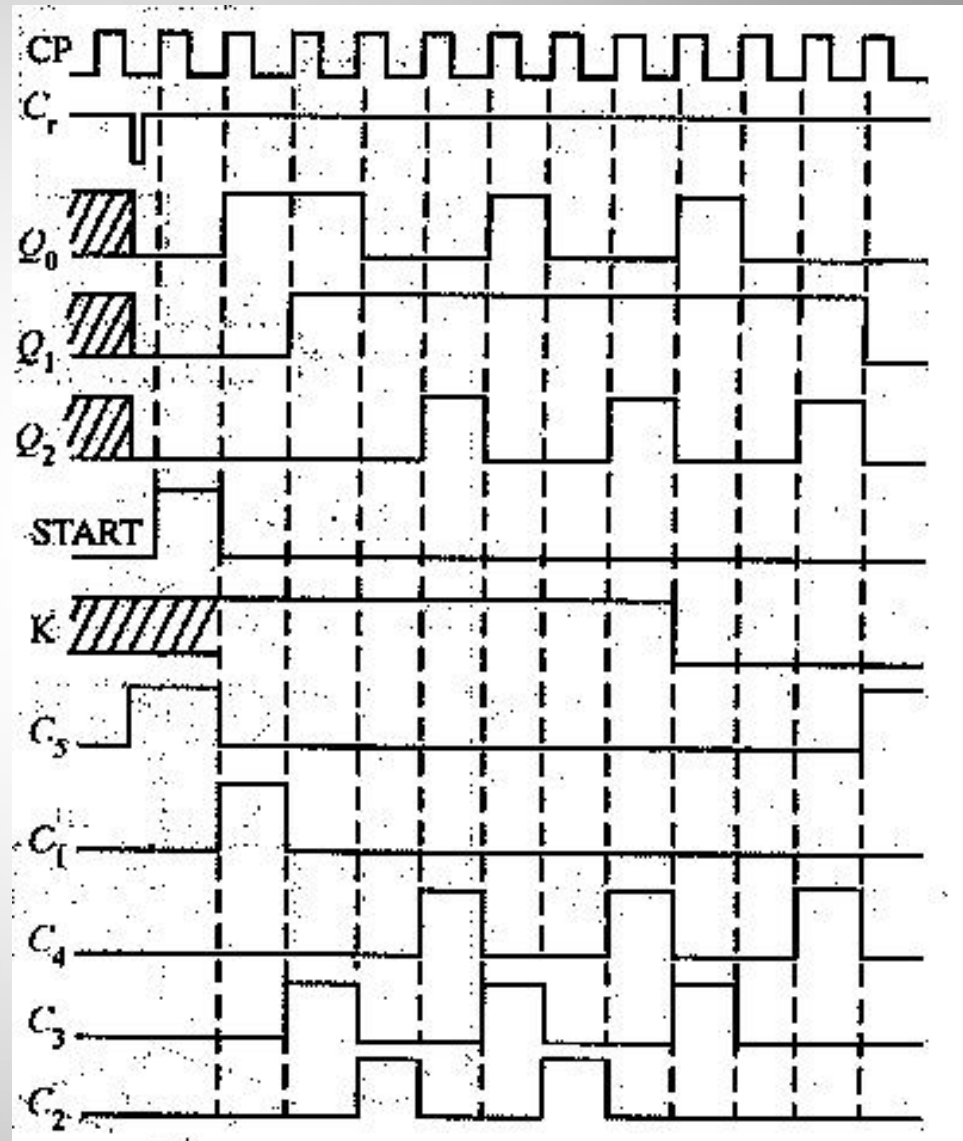
• 采用了数据选择器(MUX)构成激励函数发生器；

- 每个触发器配备一只数据选择器，这种激励电路与控制算法一一对应。
- 如果控制算法有所修改和变动，只要更改与数据选择器有关通道的输入，不必更改激励电路，这样就为控制器设计提供了方便。



5. 控制器的工作波形图

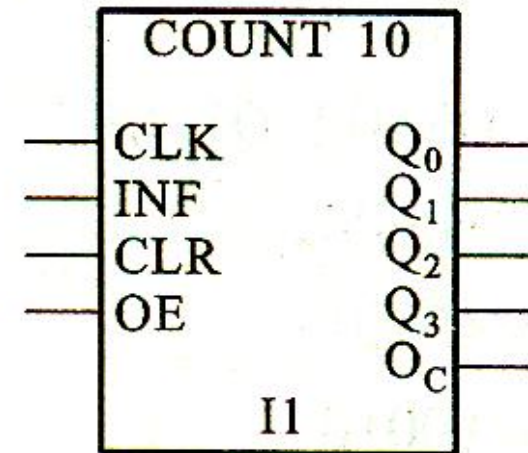
- 外部输入**START**信号保证**CP**脉冲的有效作用沿能够检查到它的变化。
- 并设定状态发生器记忆元件均响应**CP**脉冲的上升沿



【例8.4】试用状态机法设计一个异步清零的十进制加/减法可逆计数器。

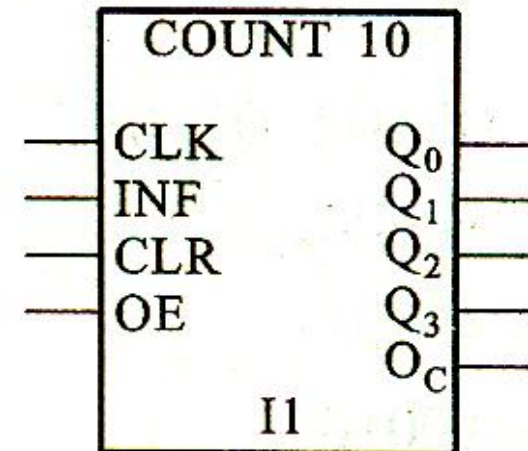
解 1. 确定任务。

- 据题意，得到计数器的设计方框图，如图所示。
 - 电路中Q0...Q3为当前计数值，
 - » 其中Q0为最低位，Q3为最高位。
 - 计数的加/减由INF信号决定：
 - » 当INF为高电平时，计数器每来一个时钟脉冲加1；
 - » 当INF为低电平时，则每来一个时钟脉冲减1。
 - 有效计数范围是0至9，
 - » 在加计数时，计数从0到9，再返回到0；
 - » 在减计数时，计数反过来，从9到0，再回到9。



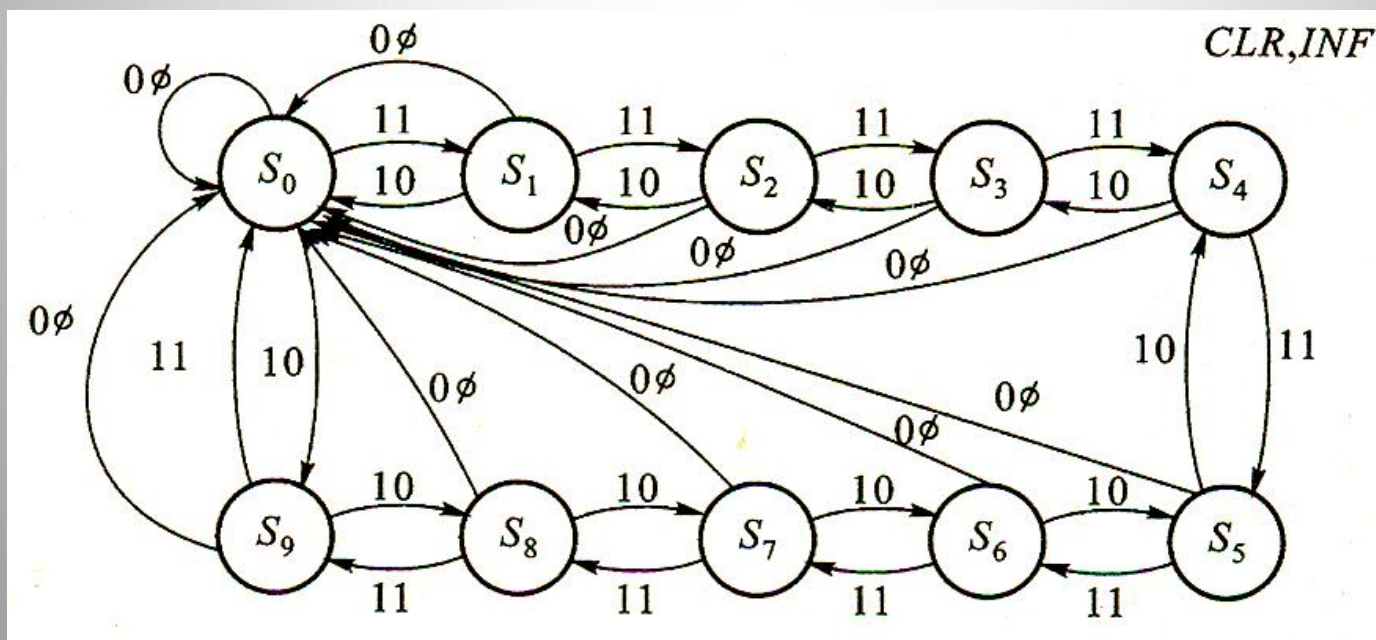
解-2

- 由于PLD的输出要经过一级缓冲器，所以这里定义使能端OE为输出允许端
 - 当OE为低电平时，输出允许；
 - 当OE为高电平时，输出呈高阻状态。
- 即使在输出被禁止时，计数器仍在时钟到来时刻进行加/减计数，
- 使能端OE控制的只是输出缓冲器，而非寄存器。
- 寄存器数值的变化与实际输出值无关。



2. 状态转移图。

- 由于存在十个可能的计数值，所以对对应要十个状态分别表示，
 - 如图中的 $s_0, \dots, s_9$ 。
- 输入信号**CLR**和**INF**决定了计数器的工作方式，
 - 计数器有加、减、清零三种工作方式，分别用11，10和00表示，其中清零工作方式中X的状态为无关项。



3. 数字电路实现。

- 10个状态根据编码原则可以用4位二进制数来表示。对应的状态机使用4个寄存器。
- 具体电路图作为课后作业。

课后作业

• 1) 查阅:

- 国际电路公司的

- » 8051, PIC, AVR单片机, ARM, MIPS, DSP等微控制器芯片的型号、类型、速度.....

- 图书馆资源: 电子器件天地, 软件

• 2) 习题: /补充讲义**P41**

- 学在浙里+[例8.4]的具体电路图实现

• 3) Project

- **Project1 (必做)**: 将讲义P31页图8.33的框图用具体型号的数字集成电路设计实现为具体电路图。(6月6日, 交报告和电路图)

- **Project2 (选做平时成绩加分)**: 用具体逻辑电路描述设计并FPGA实现一个4bit简单CPU实物。(期末停课前完成)

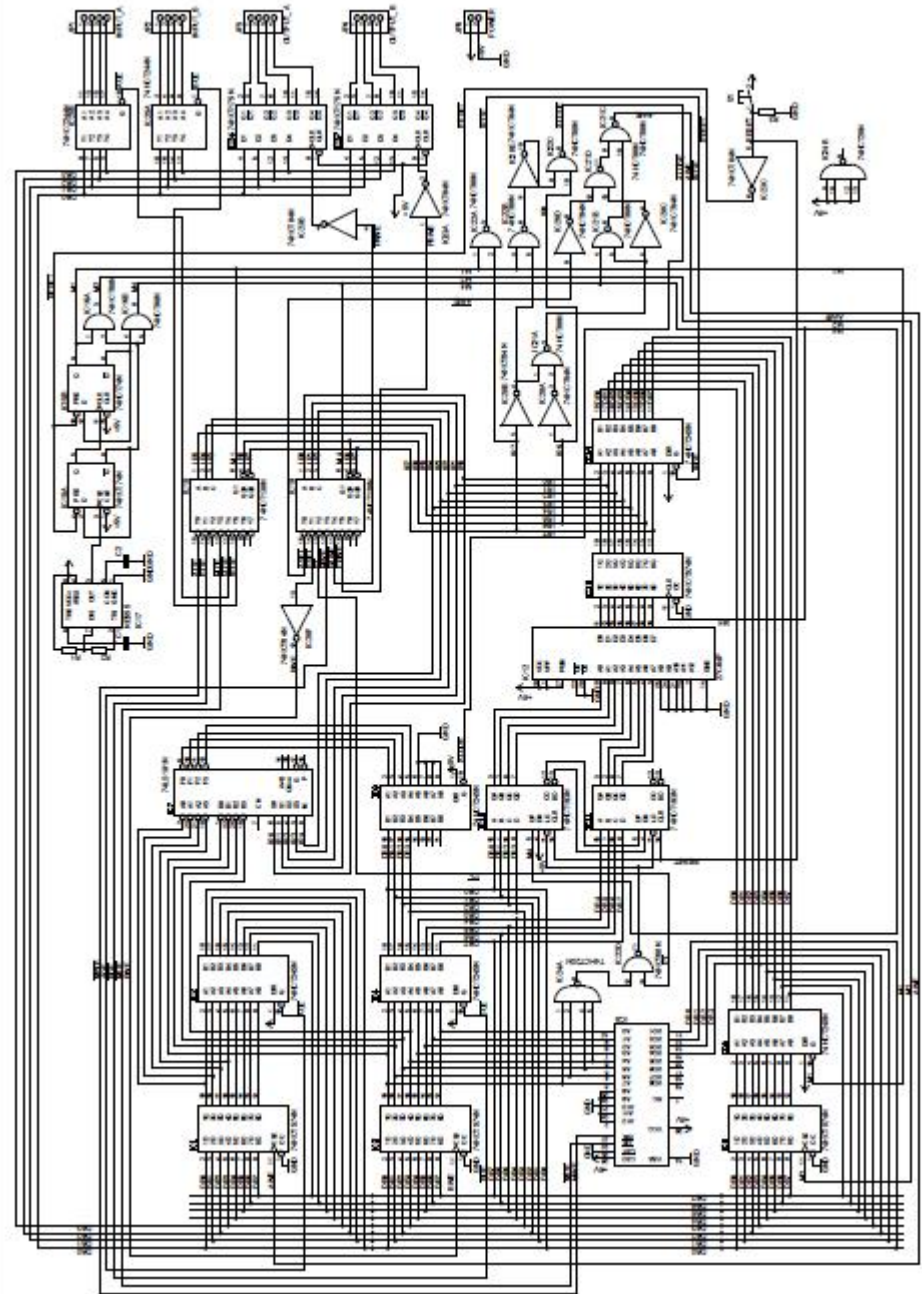
• 4) 阅读:

- 补充讲义

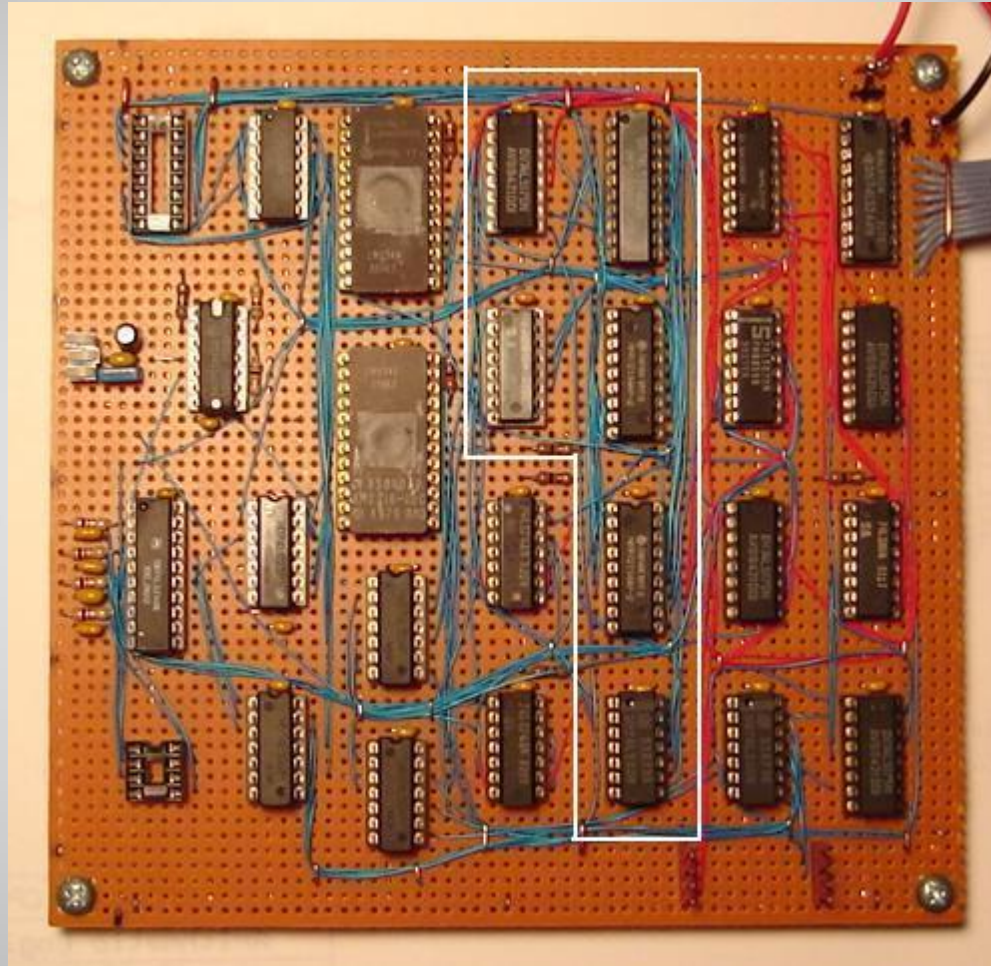
- 《数字系统设计与Verilog HDL》书

4bit Simple CPU

- ALU
- 时钟
- PC计数器
- 寄存器
- 程序存储器
- 寄存器组
- 总线



4bit Simple CPU



参考资料

- www.ttlcpu.com

新第8章 微控制器设计

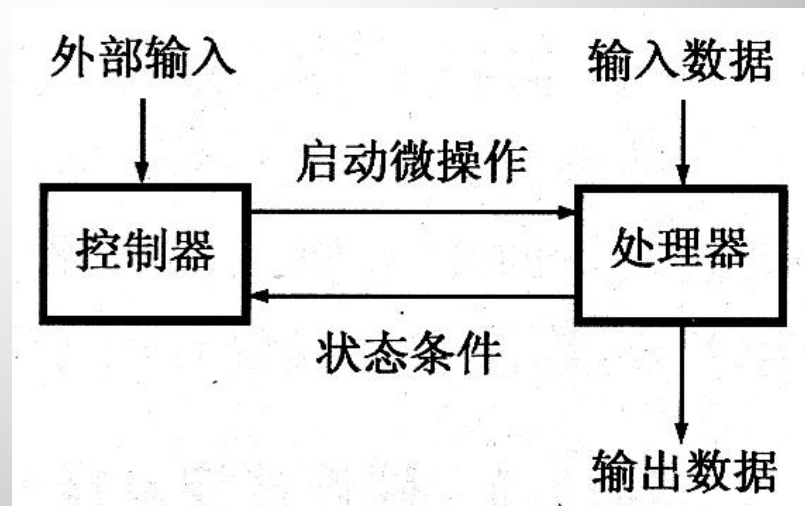
-----控制器设计

8.3. 控制器的设计

1.控制器的基本概念

- 数字系统中，各个子系统通过**数据总线**联接形成数据通路，
- 控制信号的作用
 - » 能实现数据在各子系统之间的传输、处理或存储。
- 这些控制信号就来自控制器
- **控制器本质上是一种时序网络**

• 控制器在系统中的作用



控制器的作用

控制器的作用

- 解释所接收到的各个输入信号
- 使整个系统按指定的方式工作
 - » 根据输入信号和预定的算法流程图程序

• 控制器的实现

- 控制程序可以直接由硬件实现，硬连线；
- 也可以由固件(固化的控制软件)实现，用EPROM存储；
- 由硬件直接实现的控制器设计与一般时序电路并无区别
 - 本质上看，控制器是一种时序逻辑电路子系统。

控制器设计的特点

- 控制器设计的特点

- 控制器的成本

- » 仅占系统总成本的小部分

- 控制器的性能

- » 对整个系统的工作有举足轻重的影响

- 控制器的状态----不必追求状态最简

- 首先考虑工作性能的优劣，工作是否可靠直观

- » 不必过分追求最简状态；

- 触发器的数量----也不必追求最少

- 使数字系统工作更加直观，便于监视和检查故障

- » 有时，可在控制时序中增加一些多余状态；

控制器的设计方法

- 控制器的设计方法

- 状态发生电路

- » 控制器的形式可以多种式样，但其基本设计方法有较强的规律性

- 状态单纯的“一对一”法设置触发器one-hot

- » 即一个状态设一个触发器，使控制状态单纯而明确，避免状态译码的麻烦。控制器的类型

- 小型控制器的一般设计方法

- 虽然控制器的形式多种多样，但其基本设计方法有较强的规律性。

- » 1) 计数器型控制器

- » 2) 多路选择器型控制器

- » 3) 定序型控制器

- » 4) 微码控制器 等

2. 计数器型控制器

- 直接由硬件实现的控制器
 - 其结构形式有计数器型、多路开关型、定序型等多种，适用于小型简单数字系统。
- 微程序控制器
 - 将控制程序固化在**EPROM**中，通用性强，设计简单，结构规整，适用于大型或复杂数字系统。
- **ASM**流程图设计法
 - 按给定的**ASM**流程图构造一个状态发生电路
 - » 使它具有**ASM**中所需的全部状态
 - » 能依照控制算法条件进行状态转移和条件输出
 - 由于计数器本身有许多不同的状态，因而各种形式的计数器均可以改造为控制器。

2. 计数器型控制器

- 计数器型控制器

- 计数器改造为控制器

- » 根据计数器本身不同的状态，按一定原则进行编码分配；
 - » 按照控制条件实现状态的转移；
 - » 从不同状态下的输出得到所需要的控制信号。

- 控制状态数较少的控制器

- » 可采用环形计数器作为状态发生电路，它的一个触发器对应一种控制状态。

- 当控制状态数较多时

- » 为了节省触发器数目，宜采用编码方式组成状态。

- 状态编码

- » n 个触发器最多可代表 2^n 个状态，也就是可以构成 2^n 个状态编码。

- 计数器型控制器

- » 将所要求的控制状态按一定原则进行编码分配，就可设计出一种状态计数型的控制器。

计数器型控制器框图

由三部分组成：

—计数器含有 n 个触发器

—触发器的状态

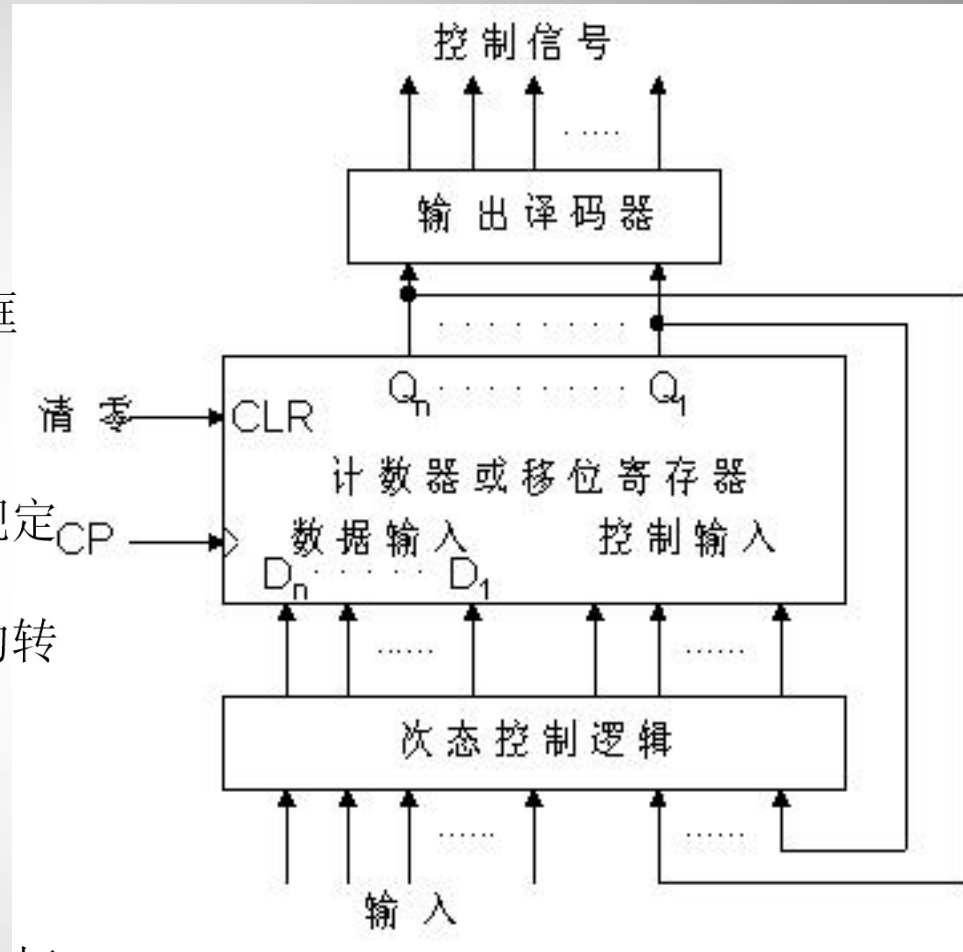
- » 状态变量以**二进制编码**的形式
- » 赋予ASM流图中的每一个状态框
- » 而条件输出框不予赋值。

—次态控制逻辑

- » 按照输入条件及状态流程图所规定的状态转移的要求设计
- » 使计数器状态按流程图所规定的转移顺序进行**计数转移**
- » 实现状态激励函数逻辑表达式。

—控制信号

- » **计数器状态经译码后输出**
- » 脉冲控制信号一般用作寄存器的打入信号，电位信号一般用作数据通路选择信号。



时间的同步和协调

- 控制信号中，有的是电位信号，有的是脉冲信号。
 - 脉冲控制信号一般用作寄存器的打入信号
 - 电位信号一般用作数据通路选择信号。
 - 保持时间上的**协调和同步**是十分重要的。
- 为此可将状态**周期**划分为 **T1** 、 **T2** 两个节拍时间：
 - 用 **T1** 驱动触发器的状态发生翻转，
 - 用 **T2** 同电位控制信号相与后发向各子系统，作为寄存器的打入控制信号。
- **电位控制信号**的持续时间是一个状态周期 (**T1 + T2**) 时间
- 而脉冲控制信号的持续时间仅为 **T2** 时间。

【例8.5】

- 图8.20为某一控制器的算法流程图，请设计一个计数器型控制器。

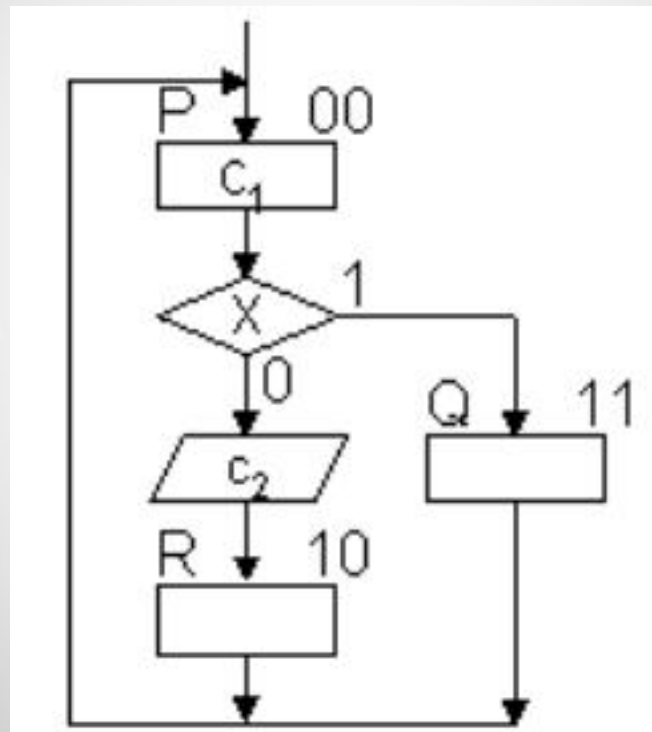


图8.20 状态已编码的ASM流程图

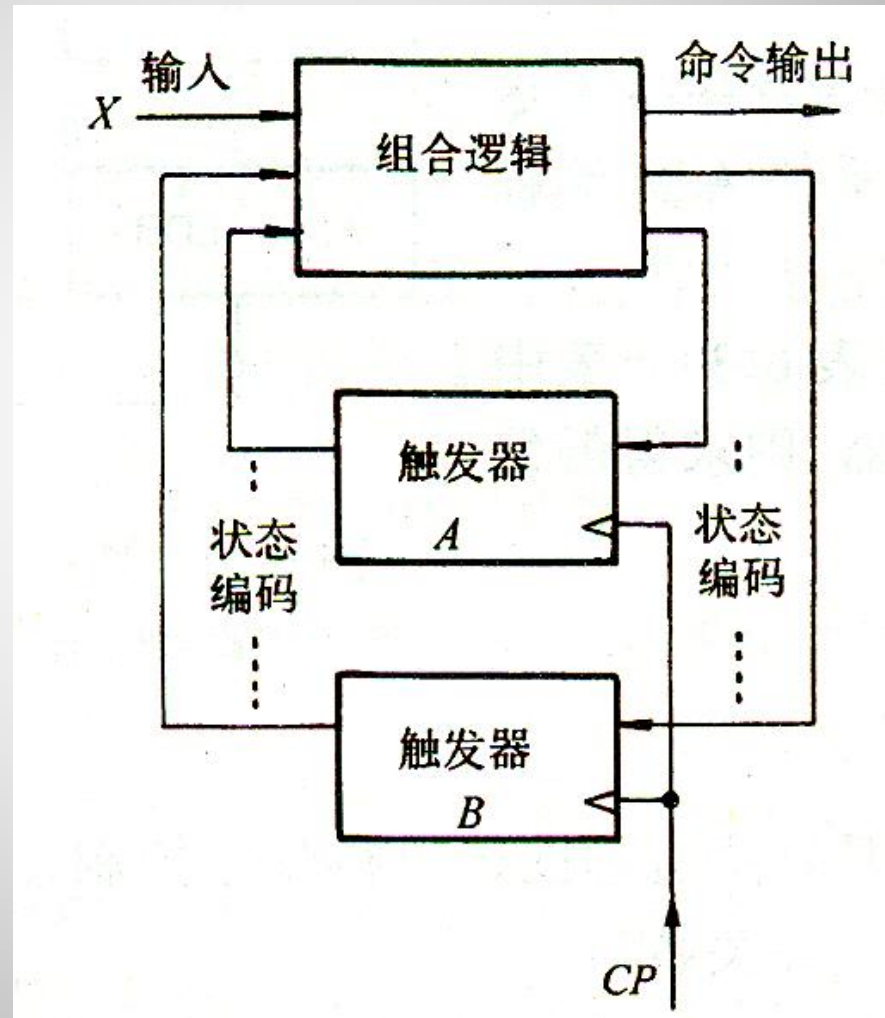
解

根据控制算法流程图可知

- ✓ 该控制器需要两个触发器，应设置两个状态变量**A**和**B**。
- ✓ 触发器此处采用D型触发器。
- 从**ASM**流程图可看出算法状态机的输入和输出参数。
 - 本例的算法状态机的输入为**X**，并有两个控制命令输出。
 - 两个触发器**A**和**B**组成图中三种编码**00**、**10**和**11**。

解

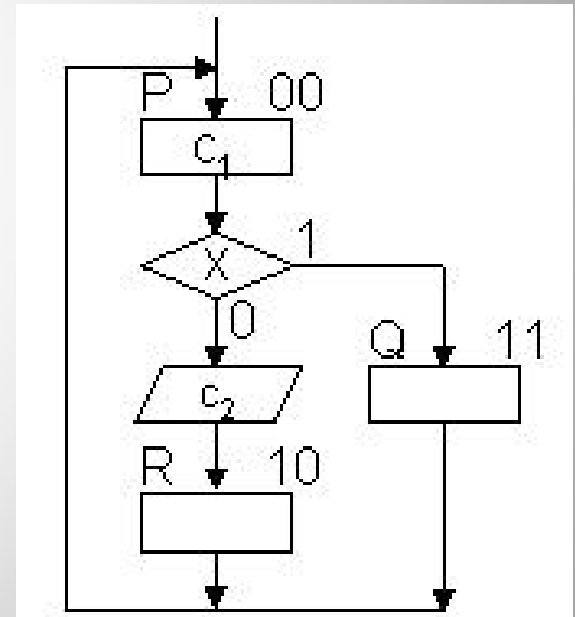
- 基本电路模型



- 次态激励函数的组合逻辑
 - ----表8.2所示。
- 全状态编码---四种不同的编码
 - 编码01状态没有用到
 - 为了避免电源刚接通时状态被锁在01的无用状态，必须使01的次态为某一有效状态
 - » 设01的次态为00

表8.2 状态激励函数真值表

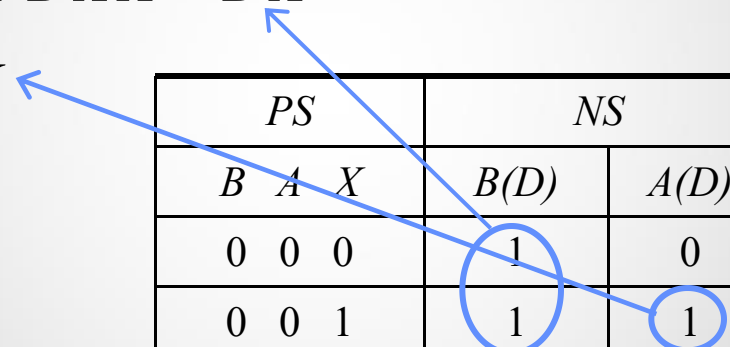
<i>PS</i>			<i>NS</i>	
<i>B</i>	<i>A</i>	<i>X</i>	<i>B(D)</i>	<i>A(D)</i>
0	0	0	1	0
0	0	1	1	1
0	1	×	0	0
1	0	×	0	0
1	1	×	0	0



- 编码方式状态发生电路
 - 所有四种不同的编码情况都考虑在内，同时解决了该电路的初态引导问题
- 根据表8.2，写出触发器A和B的次态激励函数和的表达式：

$$B(D) = \overline{B} \overline{A} X + \overline{B} \overline{A} \overline{X} = \overline{B} \overline{A}$$

$$A(D) = \overline{B} \overline{A} X$$



<i>PS</i>			<i>NS</i>	
<i>B</i>	<i>A</i>	<i>X</i>	<i>B(D)</i>	<i>A(D)</i>
0	0	0	1	0
0	0	1	1	1
0	1	×	0	0
1	0	×	0	0
1	1	×	0	0

控制信号

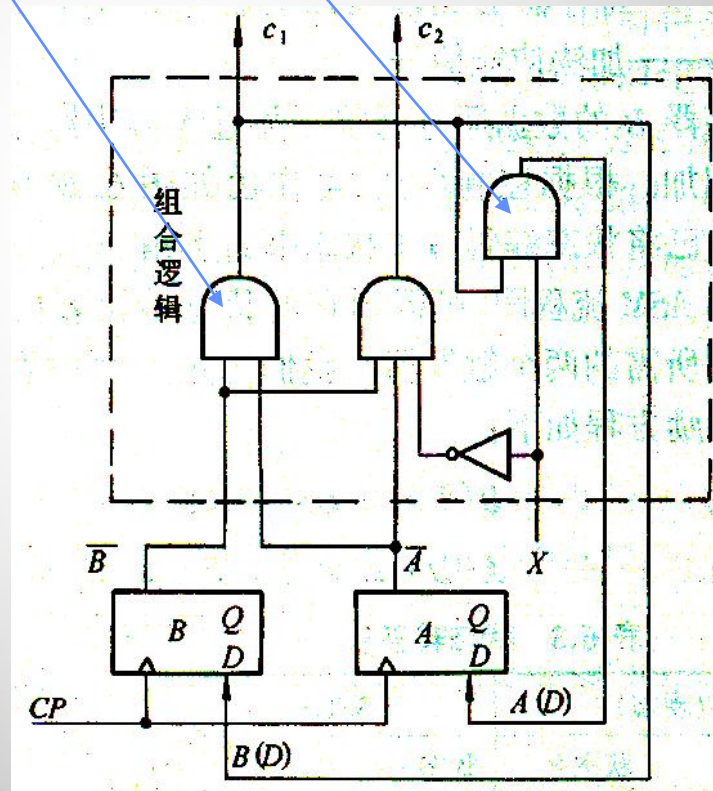
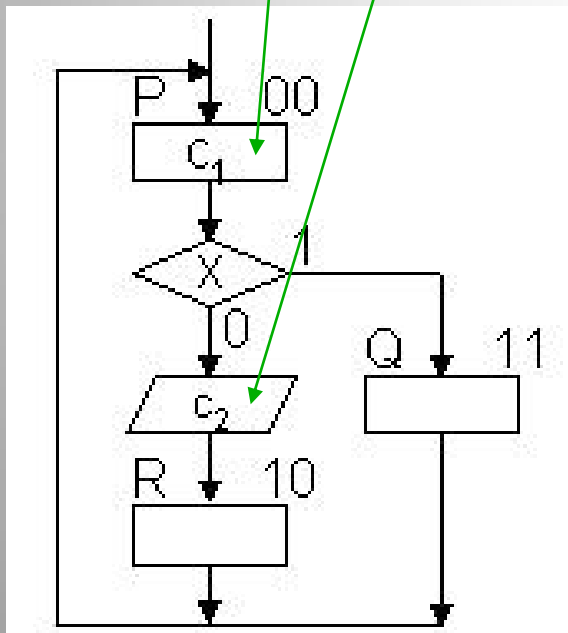
- 控制信号

 - 控制器的主要目的是产生一定的控制命令。

- $C1 = \text{状态} P = \overline{B} \overline{A}$

- $C2 = \text{状态} P \text{ 和 } X' \text{ 的“与”} = \overline{B} \overline{A} \overline{X}$

- 控制器的具体电路

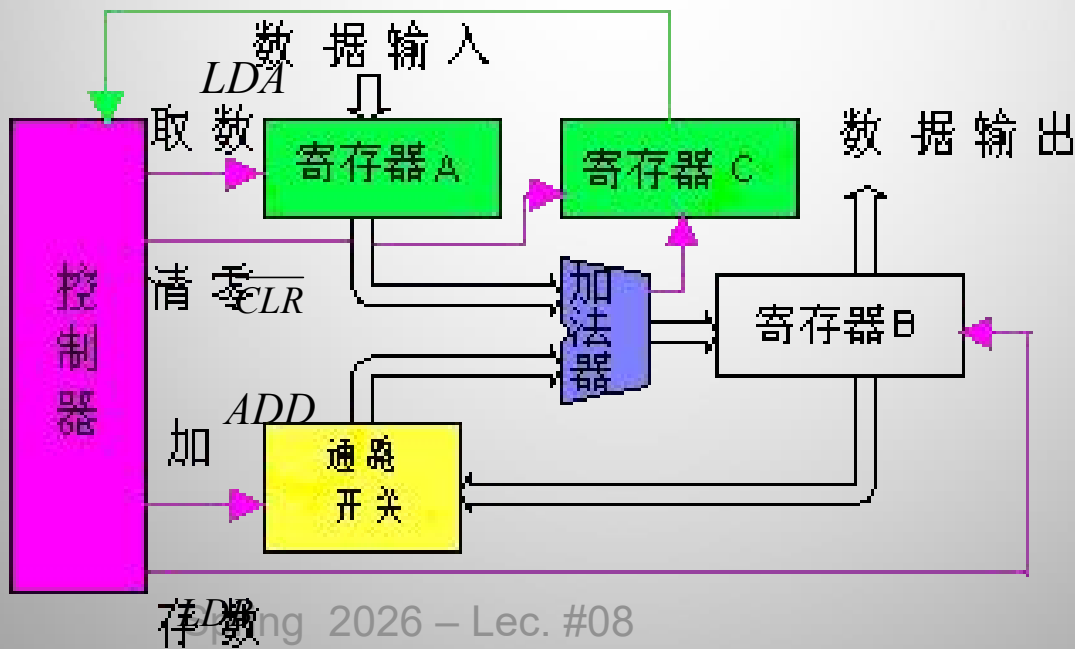


【例8.6】

一个简单的8位二进制无符号数并行加法运算器，数据从存储器通过同一个总线输入，控制算法分四步：

- 1) 溢出标志寄存器C清零，先取被加数至寄存器A；
- 2) 将A中被加数送寄存器B；
- 3) 后取加数至寄存器A；
- 4) 将A与B中的数相加，结果存于B，且1位进位标志存放于寄存器C。

请设计该控制器，要求采用计数器型。假设状态周期 $T = T_1 + T_2$ 计数器状态变化发生在 T_1 时序，控制信号LDA，LDB发生在 T_2 时序。

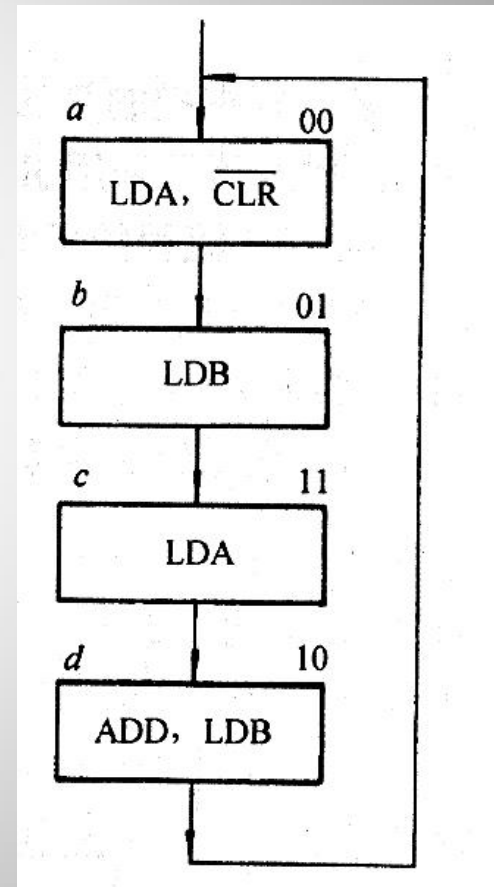
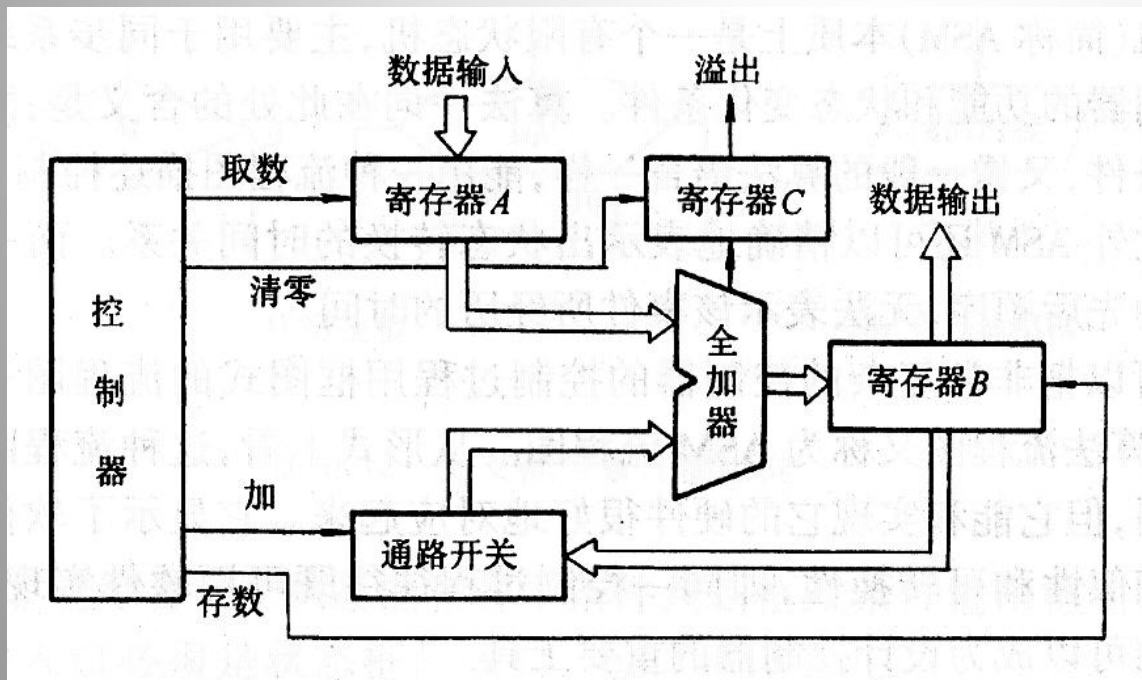


【例8.6】

解 并行加法运算器的控制器的设计中，有**四步**算法操作完成两个数相加，控制器有**四个**控制状态。

- 设四个状态的名字为**a、b、c、d**,
- **微操作**命令命名为
 - $\overline{CLR}$ — 寄存器**C**清零;
 - LDA — 寄存器**A**接受数据;
 - LDB — 寄存器**B**接收从加法器送来的数据;
 - ADD — 加法使能信号。
- 被加数分**2步**输入到寄存器**B**;
- 寄存器**B**的数据通过**通路开关**进入加法器，并与寄存器**A**送来的数相加。

- 状态 2位二进制编码 $a=00, b=01, c=10, d=11$
- 根据控制算法，可作出**ASM**流程图。
 - 图中已将状态编码写于状态框右上角。



- 根据ASM流程图，可得到状态转换表(见表8.3)。

– 表中A和B是所需的两个触发器。

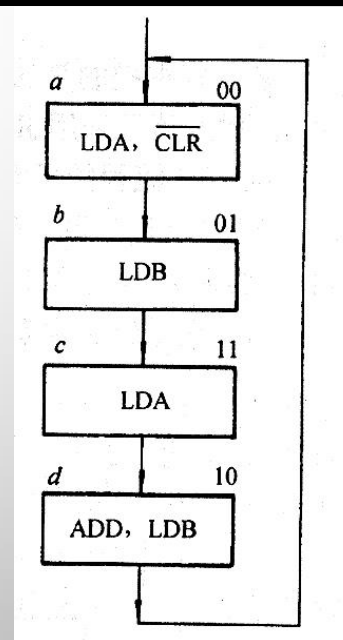
- 选用D型触发器，则求得触发器次态激励方程：

$$B(D) = \overline{B}A + BA = A$$

$$A(D) = \overline{B}\overline{A} + \overline{B}A = \overline{B}$$

表8.3 状态转移表

PS(现态)		NS(次态)			
B	A	状态名	状态名	B(D)	A(D)
0	0	a	b	0	1
0	1	b	c	1	1
1	1	c	d	1	0
1	0	d	a	0	0



- 从算法流程图得到各控制信号的逻辑函数关系

$$LDA = (\overline{B} \overline{A} + BA)T_2$$

$$LDB = (\overline{B} \overline{A} + \overline{B} A)T_2 = (B \oplus A)T_2$$

$$\overline{CLR} = \overline{B} \overline{A}$$

$$ADD = B \overline{A}$$

T1:时钟节拍的前半部分用于产生结果;
控制信号LDA, LDB发生在T₂时序

T2: 时钟节拍的后半部分(上升沿需要)

— 注:

- » 其中LDA、LDB为脉冲控制信号
- » $\overline{CLR}$ 、ADD为电位控制信号
- » 控制信号LDA、LDB在上升沿有效
- » 电位控制信号持续时间应与状态周期T相同

运算器与控制器电路图

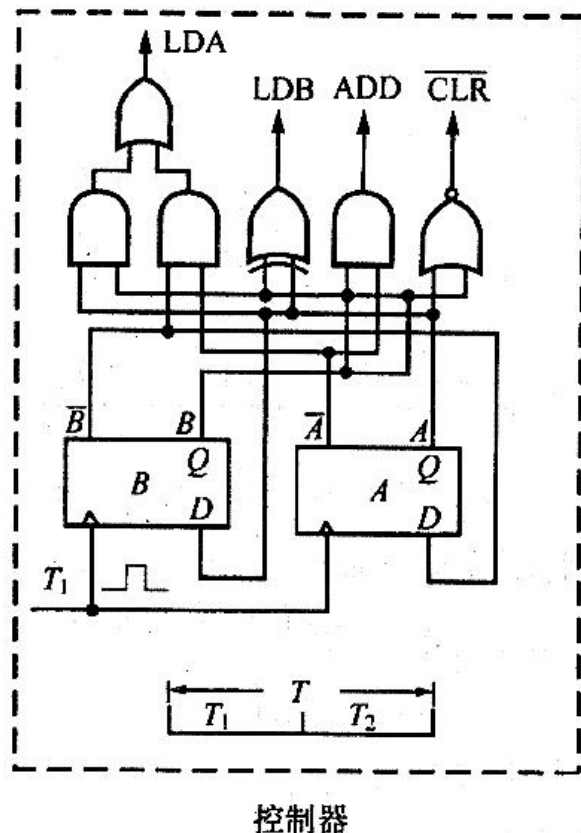
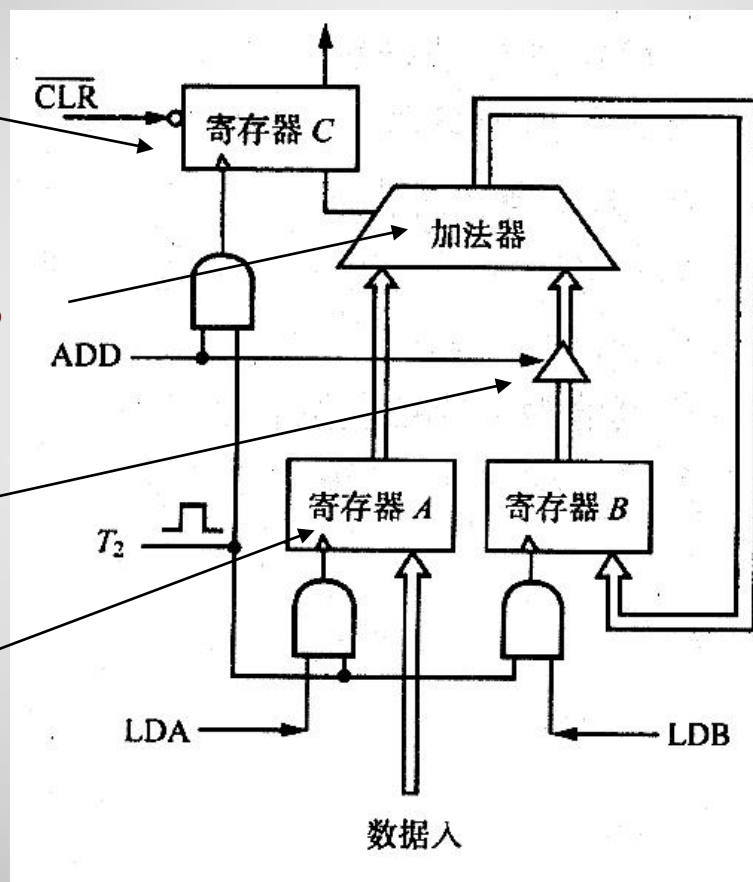
- 数据通道

寄存器C选用74LS74

全加器选用2片74LS283

三态缓冲器采用74LS244

寄存器A和B选用 74LS273八D触发器



注意，图中右边的控制信号是电位信号，为了加入时间因素 T_2 左边电路中增加了少量的与门以产生脉冲控制信号

小结

- 计数器型控制器的设计方法是一种经典方法。
- 缺点
 - 硬件逻辑电路与控制算法间没有明显的对应关系
 - 算法流程图有微小变动，都要重新逐一计算生成次态激励函数
 - » 会牵动全局
 - 这种设计方法不“清晰明确”。
- 我们希望设计过程标准化
 - 次态函数生成电路与ASM流图有清晰的对应关系。

3.多路选择器型控制器

- 多路选择器的任务

- 按控制算法要求，为其对应的触发器生成次态激励函数
- 作为控制器状态触发器的次态激励函数的生成电路

- 采用MUX特点

- 可以实现所有最小项函数
- 能使控制器的设计过程标准化
- 可以使整个控制器电路“清晰明确”

- 多路选择器的电路特点

- 所有MUX输出端的组合就是控制器次态（NS）的编码；
- MUX的输入端则是对应所有现态（PS）的状态转移条件；
- MUX的地址码受现态PS控制。
- 多路选择器应有足够的输入端，所有的状态变量都有其相应的输入端

这样对应某个PS状态值，MUX输出一个NS状态值，从而使触发器的次态按NS值进行改变。之后又以NS值作为PS值控制MUX的地址码，输出新的NS值。

多路选择器型控制器设计步骤

- 任务分析

- 按**ASM**控制算法要求
- 给**MUX**的每一个输入端提供适当的输入值
- 使之能给出次态所要求的逻辑值

- 设计思路和步骤

- 给**ASM**图中的每一个二进制编码状态赋予一个十进制数码，和多路选择器的输入端编码相对应。
- 按**ASM**流程图建立起状态转换表
- 求出**MUX**的各个数据输入端的函数值；
- 按**ASM**流程图写出控制命令的逻辑表达式
- 画出具体电路图，完成控制器的设计。

【例8.7】

- 设计一个多路选择器型控制器，实现图8.20的控制算法。

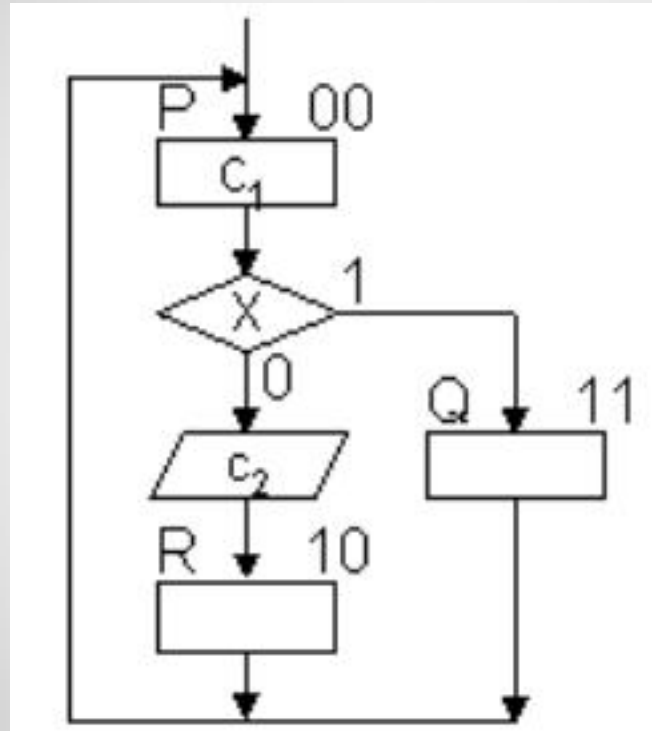


图8.20 状态已编码的ASM流程图

解：

- 根据算法流程图

- 作出状态转移数据表，如表8.4所示。
- 状态转移数据表比通常的状态转移表多了一栏——转换条件
- 转换条件栏的数据或表达式，对应的多路选择器的数据输入值

表8.4 图8.20的状态转移数据表

<i>PS</i> (现态)		<i>NS</i> (次态)			转换条件
编码	状态名	状态名	<i>B</i>	<i>A</i>	
0 (00)	P	R	1	0	X=0
		Q	1	1	X=1
2 (10)	R	P	0	0	0
3 (11)	Q	P	0	0	0
1 (01)	—	P	0	0	0

电路设计

• 电路设计

- 选用两个D型触发器FA和FB
- 需要两个4位数据选择器
- 多路选择器的输出就是触发器的输入，也是触发器的次态激励函数
- 多路选择器的控制端分别与触发器的Q输出端相连
 - Q_A 和 Q_B
- 根据状态转移表
 - 将次态变量中真值为1的各项按转换条件写出
- 得到多路选择方程

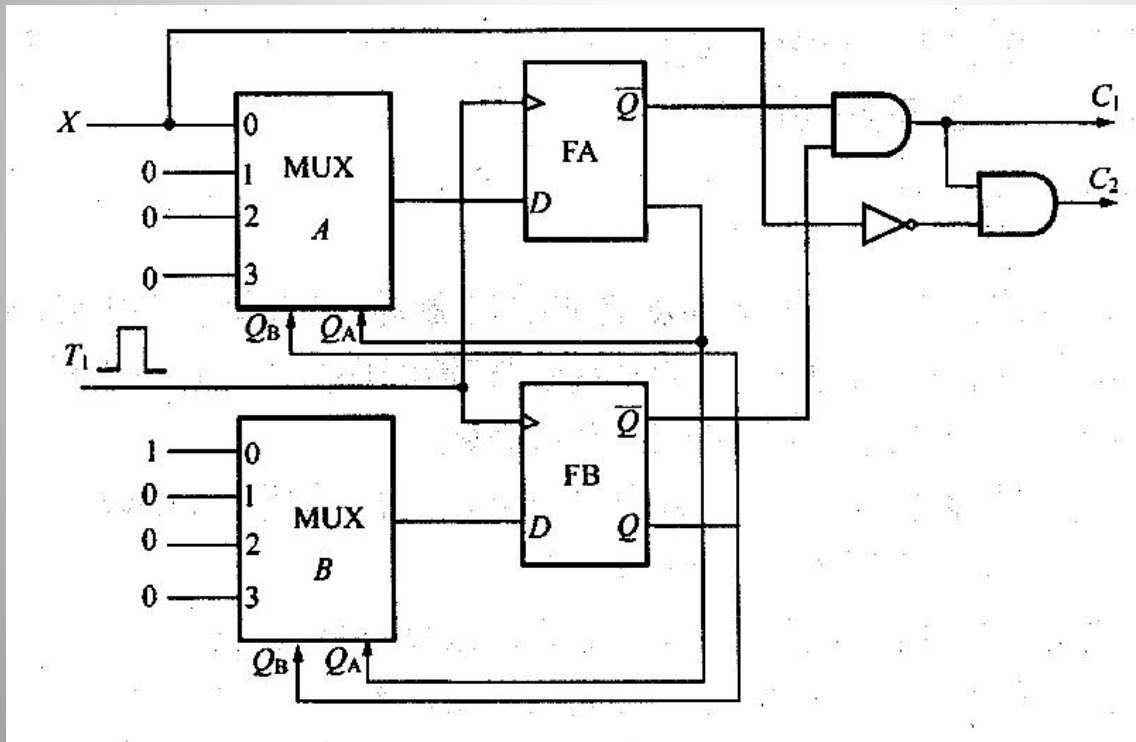
$$\begin{array}{ll} MUXA(0) = X & MUXB(0) = X + \overline{X} = 1 \\ MUXA(2) = 0 & MUXB(2) = 0 \\ MUXA(3) = 0 & MUXB(3) = 0 \\ MUXA(1) = 0 & MUXB(1) = 0 \end{array}$$

用PS作为MUX的地址输入，用转换条件C作为MUX的数据输入

MUX型控制器电路

- **MUX型控制器电路**

- 其控制信号可从ASM流程图的输出条件直接得到
- 利用触发器的反码端信号得以简化



设计总结

- 设计过程相当规正
 - 用MUX来设计控制器的方法简单易行
- 设计容易修改
 - 当ASM流程图有变化时，只需对电路中相应的MUX输入作适当改变就行了，不至于牵动全局，重新计算各状态激励函数
 - 器件基本上不受算法流程图变化的影响
- 一旦熟悉了这种设计方法后，可以不必列出次态函数计算式，而直接从控制算法流程图决定多路选择器的各个输入值。

【例8.8】

- 图8.25 (a)为某控制器的ASM流程图，试用MUX来实现该控制器。

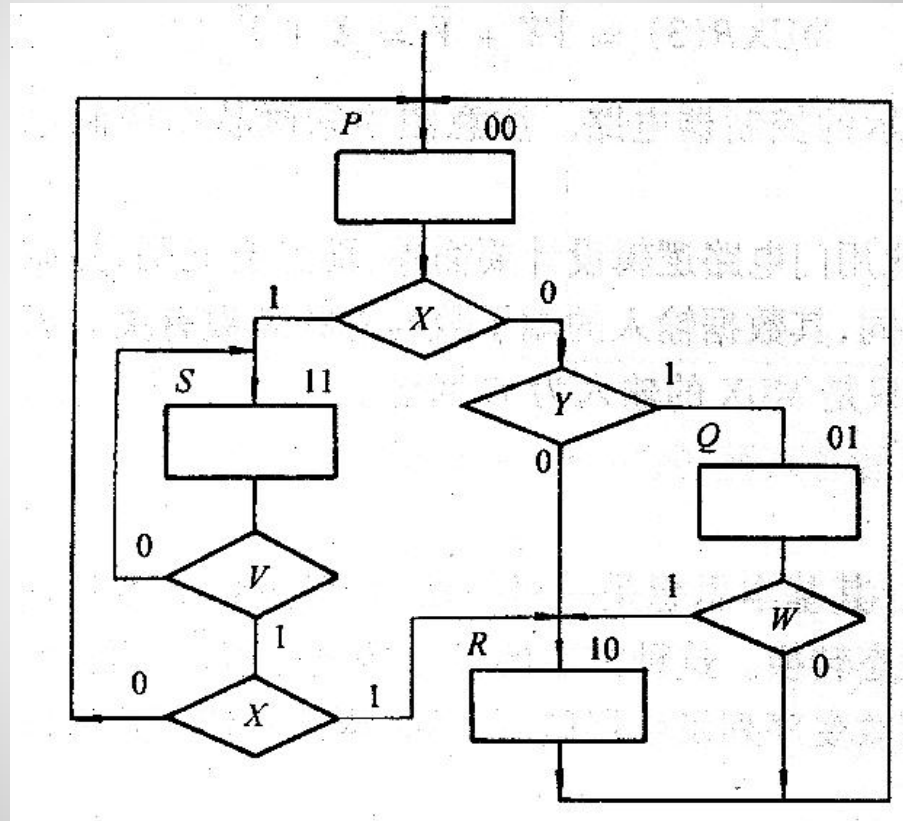


图7.25(a) 例7.8的MUX型控制器ASM流程图

解

- 由算法流程图得到状态转移数据表

- 表8.5所示
- 表中A和B为状态变量

表8.5 图8.25 ASM的状态转移数据表

- 控制器电路

- 共需两个触发器
- 两个4位多路转换器

PS(现态)		NS(次态)			转换条件
编码	状态名	状态名	B	A	
0 (00)	P	S	1	1	X
		Q	0	1	$\overline{X}Y$
		R	1	0	$\overline{X}\overline{Y}$
1 (01)	Q	P	0	0	$\overline{W}$
		R	1	0	W
2 (10)	R	P	0	0	–
3 (11)	S	P	0	0	$V\overline{X}$
		R	1	0	VX
		S	1	1	$\overline{V}$

求解

- **MUX的数据输入**

- 根据状态转移数据表，可以求得各个状态的转换条件。

- **得到多路选择方程**

$$MUXA(0) = X + \overline{X}Y = X + Y \quad MUXB(0) = X + \overline{X} \overline{Y} = X + \overline{Y}$$

$$MUXA(1) = 0$$

$$MUXB(1) = W$$

$$MUXA(2) = 0$$

$$MUXB(2) = 0$$

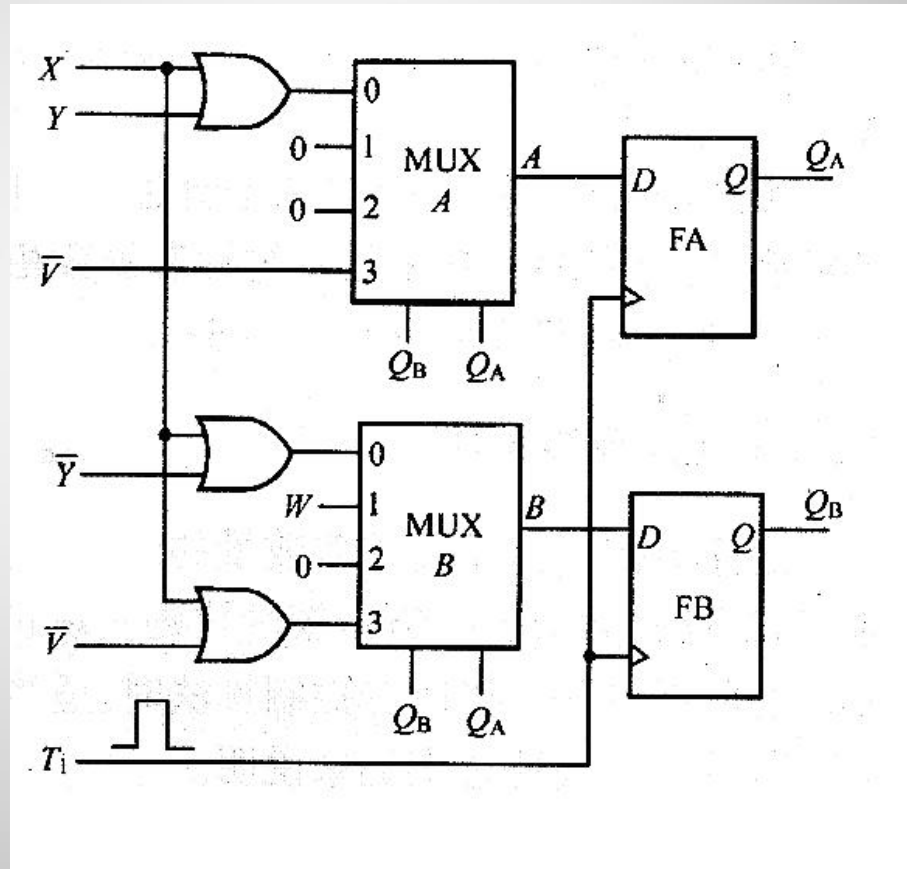
$$MUXA(3) = \overline{V}$$

$$MUXB(3) = VX + \overline{V} = X + \overline{V}$$

根据状态转移表，利用 $NS = \sum PS \cdot C$ 公式

控制器电路

— 注：图中未画状态译码电路。



小结

- 小结

- 采用**MUX**来设计控制器比用门电路逻辑设计要简单
- 有规则，控制关系明确
- 所需的**MUX**数量与触发器数相同
- 其数据输入的端子数与总状态数有关
- 事实上，这种方法仍采用公式，只是**MUX**的输入为**C**而已。

$$NS = \sum PS \cdot C$$

4. 定序型控制器

- 定序型控制器需要较多数量的触发器
- 基本思想
 - 一对一法
 - ASM流程图中有多少个状态，则使用多少个触发器
 - » 即触发器的数目代表了状态数
 - 依赖一组最新的代码实现状态转换
- 目标
 - 设计简单
- 特点
 - 避免了状态分配的麻烦，
 - 而且也不需要状态译码

状态	触发器代码		
	S ₁	S ₂	S ₃
S ₁	1	0	0
S ₂	0	1	0
S ₃	0	0	0

当改变控制算法时，适当改变序列发生器即可，不必推倒一切而重新设计控制器。

定序型控制器的设计步骤

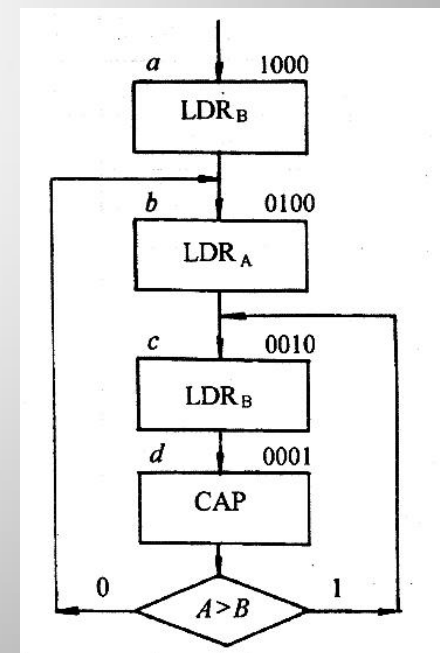
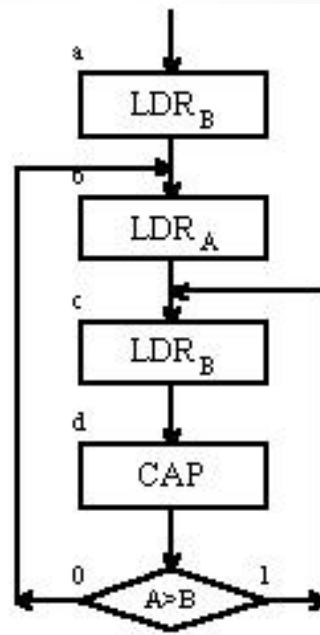
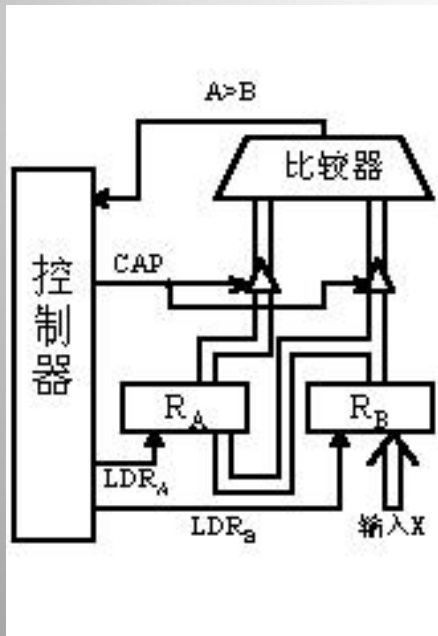
定序型控制器的设计步骤基本上与计数器型、**MUX**型相同：

- （1）根据课题要求划分子系统；
- （2）设计控制算法，画出**ASM**流程图；
- （3）按一对一法，对**ASM**流程图进行状态编码；
- （4）由**ASM** 流程图，列出状态转移表；
- （5）写出各个触发器的次态激励函数表达式；
- （6）按**ASM**流程图，写出控制信号表达式；
- （7）画出控制器的具体电路图。

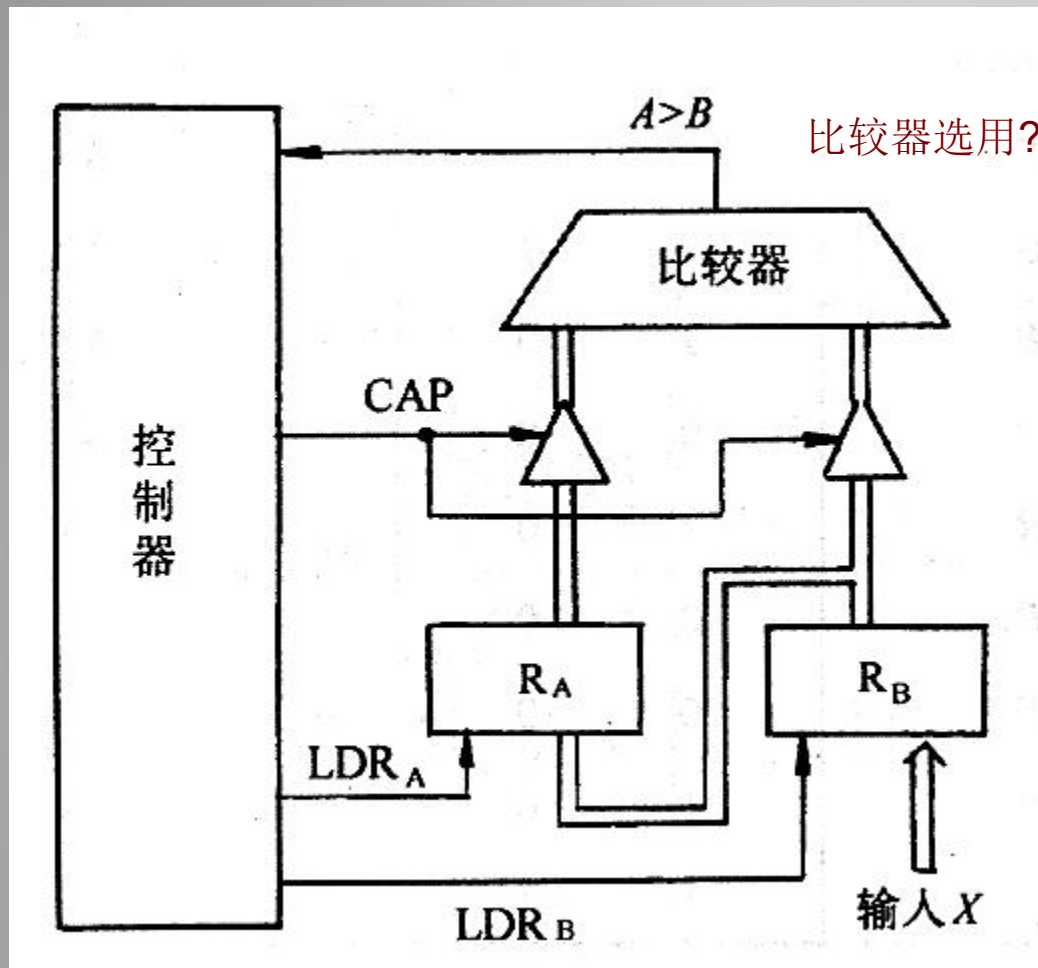
【例8.9】

有一个数字比较系统，它能连续对两个二进制数据进行比较，操作过程如下：先将两个数存入寄存器 R_A 和 R_B ，然后进行比较，最后将大数移入寄存器 R_A 中。其方框图和ASM流程图见图8.26所示。

- 其中X为输入信号， LDR_A, LDR_B 为存数控制信号，CAP是三态门使能控制信号， $(A>B)$ 是比较器输出信号。
- 请设计定序型控制器。假设状态变化发生在 T_1 时序，打入寄存器操作发生在 T_2 时序，状态周期 $T = T_1 + T_2$ 。



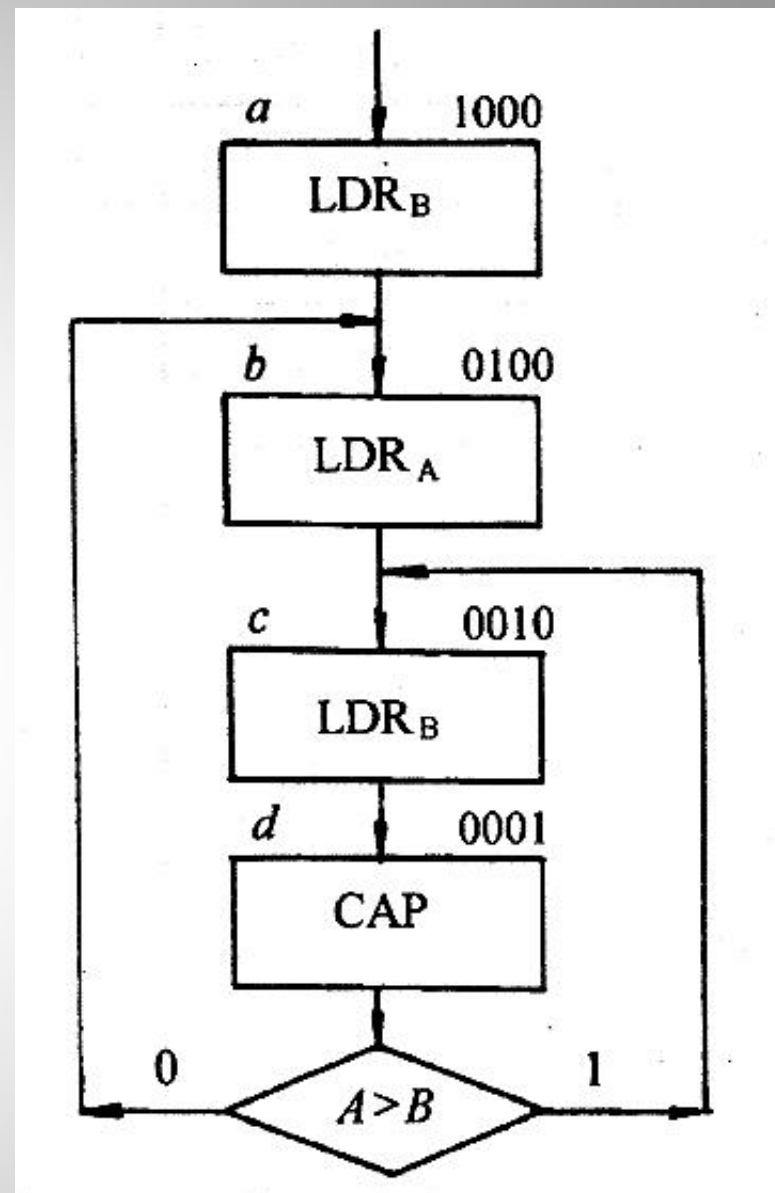
放大图



寄存器A和B选用
74LS273八D触发器

三态缓冲器采用74LS244

(a)方框图



(b)ASM流程图

解

- 定序型控制器采用“一对一法”进行设计
 - 即一个状态使用一个D触发器，共4触发器
- 首先对ASM流程图进行编码(写于右上角)。
- 表8.6列出了状态转移真值表。

表8.6 状态转移真值表

<i>PS</i> (现态)	<i>NS</i> (次态)	转换条件 <i>C</i>
$Q_a \ Q_b \ Q_c \ Q_d$	$Q_a(D) \ Q_b(D) \ Q_c(D) \ Q_d(D)$	
<i>1 0 0 0</i>	<i>0 1 0 0</i>	初始化置"1" Q_a
<i>0 1 0 0</i>	<i>0 0 1 0</i>	-
<i>0 0 1 0</i>	<i>0 0 0 1</i>	-
<i>0 0 0 1</i>	<i>0 0 1 0</i>	<i>A > B</i>
	<i>0 1 0 0</i>	$\overline{A > B}$

状态

- 状态与触发器是一一对应的
- 各触发器的次态激励方程可按 $NS = \Sigma PS \cdot C$ 公式写出

$$Q_a(D) = 0$$

$$Q_b(D) = Q_a + \overline{(A > B)} \cdot Q_d$$

$$Q_c(D) = Q_b + (A > B) \cdot Q_d$$

$$Q_d(D) = Q_c$$

- 控制信号逻辑表达式

$$LDR_B = (Q_a + Q_c) \cdot T_2$$

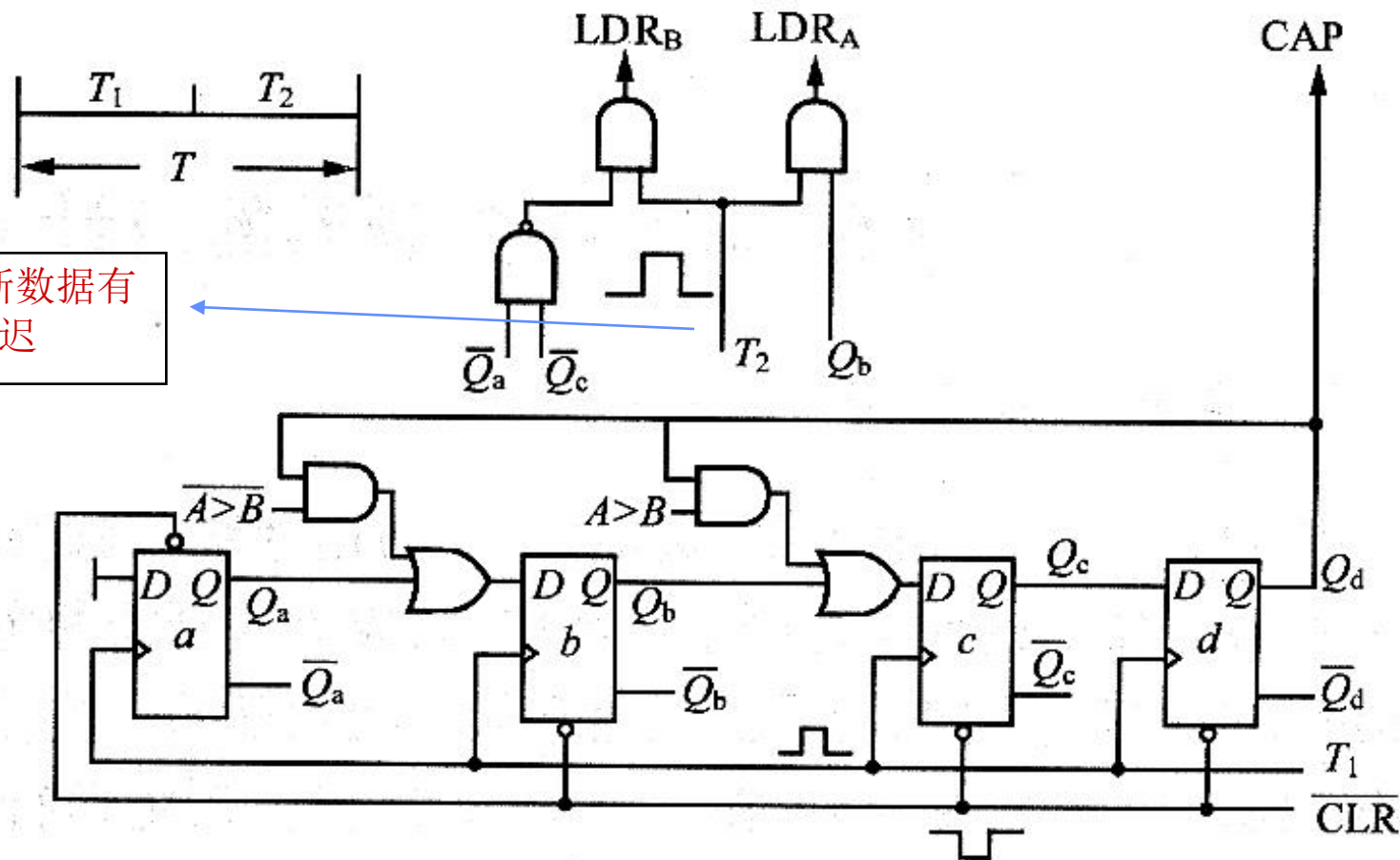
$$LDR_A = Q_b \cdot T_2$$

$$CAP = Q_d$$

定序型控制器的逻辑电路图

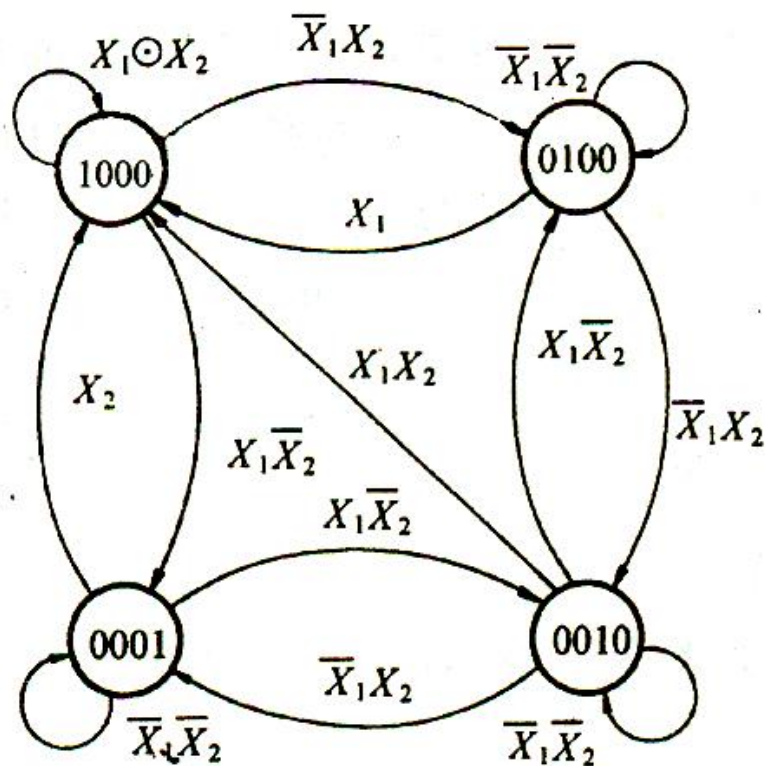
- 为了直观，驱动电路实现直接采用了与或门的形式
- LDR_B 和 LDR_A 控制信号用与非门和与门实现，**CAP** 信号不需要译码，直接从 Q_d 端引出。
- 打入控制信号和 T_2 相与，用 T_1 上升沿改变触发器状态。

状态改变到新数据有效, 有时间延迟



【例8.10】

- 图8.28所示某控制器的状态图及状态转移表，请用数字器件设计一个定序型控制器(不考虑控制信号译码电路)。



状态转移表

PS(现态)				NS(次态)				转换 条件
Q_A	Q_B	Q_C	Q_D	Q_A	Q_B	Q_C	Q_D	
1	0	0	0	1	0	0	0	$X_1 \odot X_2$
				0	1	0	0	$\bar{X}_1 X_2$
				0	0	0	1	$X_1 \bar{X}_2$
0	1	0	0	1	0	0	0	X_1
				0	1	0	0	$\bar{X}_1 \bar{X}_2$
				0	0	1	0	$\bar{X}_1 X_2$
0	0	1	0	1	0	0	0	$X_1 X_2$
				0	1	0	0	$X_1 \bar{X}_2$
				0	0	1	0	$\bar{X}_1 \bar{X}_2$
				0	0	0	1	$\bar{X}_1 X_2$
0	0	0	1	1	0	0	0	X_2
				0	0	1	0	$X_1 \bar{X}_2$
				0	0	0	1	$\bar{X}_1 \bar{X}_2$

图8.28 一对一法表示的状态图和状态表

解

- 根据状态转移表，利用公式 $NS = \sum PS \cdot C$ ，可以直接写出 Q_A 、 Q_B 、 Q_C 、 Q_D 四个触发器的次态激励方程表达式：

$$Q_A = Q_A(X_1 \odot X_2) + Q_B X_1 + Q_C X_1 X_2 + Q_D X_2$$

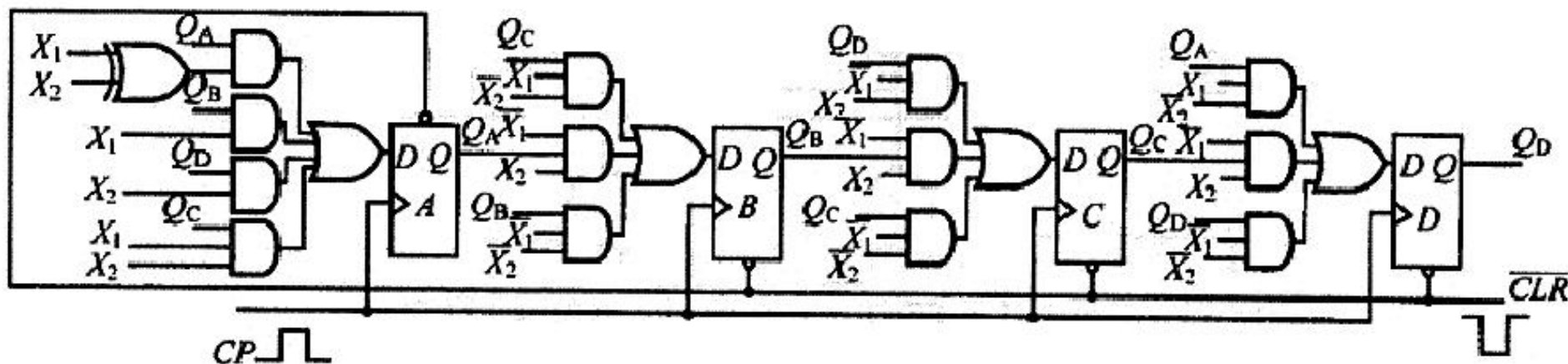
$$Q_B = Q_A \overline{X_1} X_2 + Q_B X_1 + Q_C \overline{X_1} \overline{X_2}$$

$$Q_C = Q_B \overline{X_1} X_2 + Q_C \overline{X_1} \overline{X_2} + Q_D \overline{X_1} X_2$$

$$Q_D = Q_C \overline{X_1} X_2 + Q_D \overline{X_1} \overline{X_2} + Q_A X_1 \overline{X_2}$$

一对一法

- 画出“一对一法”实现的四状态时序机的逻辑图



“一对一法”实现的四状态时序机逻辑图

- 这些方程采用与-或表达式
- 它也适合可编程器件实现
 - 可直接采用状态图和Verilog硬件描述语言等来进行设计。

小结

- 定序型控制器
 - 一对一法设计
 - 触发器的数目与状态数一样多
- 特点
 - 设计简单
 - 不需要状态译码
 - 当改变控制算法时，适当改变序列发生器即可
 - » 不必推倒一切而重新设计控制器
 - 适合状态数不是太多的控制器
- 思考：有没有更通用、更易修改变化的控制器设计方法？

课后作业

- 1) 查阅:
 - 国际电路公司的
 - » 8051, PIC, AVR单片机, ARM, MIPS, DSP等微控制器芯片的型号、类型、速度.....
 - 图书馆资源: 电子器件天地, 软件
 - ftp://10.13.71.58
- 2) 习题:
 - 学在浙大/补充讲义P43; +(例8.10) 用Verilog硬件描述语言练习;
- 3) 阅读:
 - 补充讲义
 - 《数字系统设计与Verilog HDL》书
 - 复习时序电路设计方法

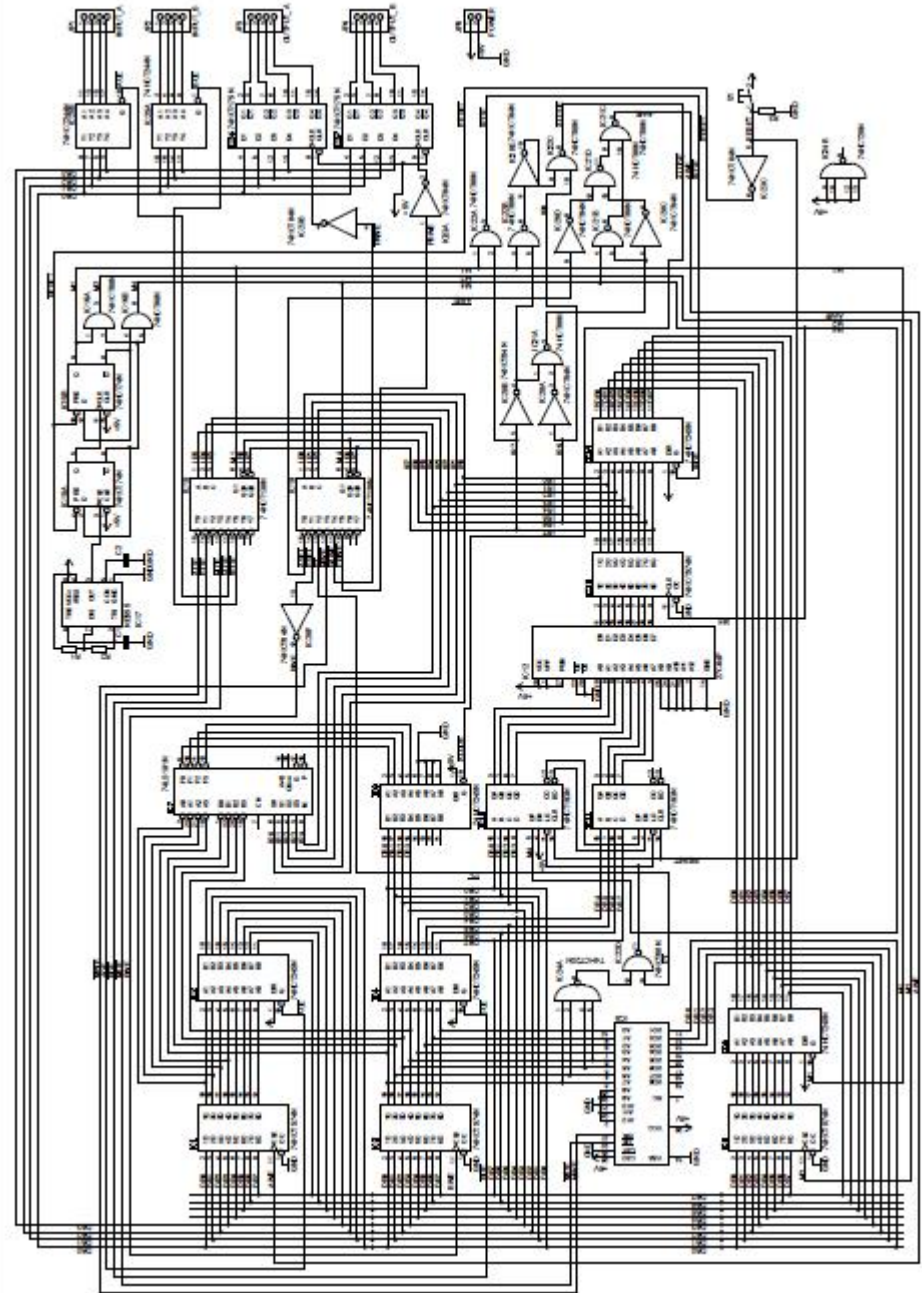
课后作业

• 4) Project

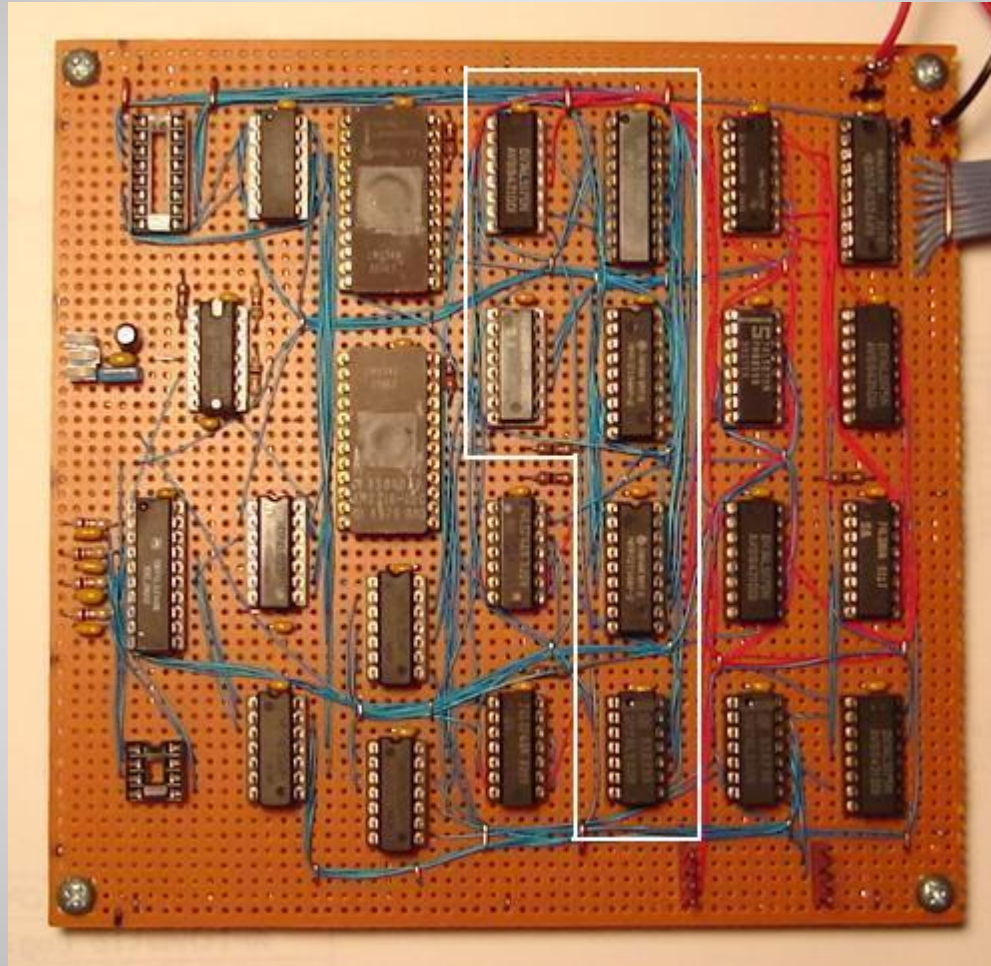
- **Project1 (必做)**: 将讲义P31页图8.33的框图用具体型号的数字集成电路设计实现为具体电路图。(交报告和电路图)
- **Project2 (选做平时成绩加分)**: 用具体集成电路设计并硬件实现一个4bit简单CPU实物。(期末停课一周前完成)

4bit Simple CPU

- ALU
- 时钟
- PC计数器
- 寄存器
- 程序存储器
- 寄存器组
- 总线



4bit Simple CPU



参考资料

- www.ttlcpu.com